DevSecOps and security

Build security into delivery: threat modeling, secrets, pipeline and supply chain checks, container and Kubernetes hardening, cloud identity and incident response.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. The DevSecOps mindset and threat modeling (15 min)
2. Secrets management (15 min)
3. Securing the delivery pipeline and supply chain (18 min)
4. Container and image security (15 min)
5. Kubernetes security (18 min)
6. Cloud security, identity and infrastructure as code (18 min)
7. Vulnerability management and security incidents (15 min)
8. Identity, authentication and zero trust (16 min)
9. Application security for DevOps engineers (16 min)
10. Policy as code, compliance and audit evidence (15 min)
11. Network, edge and DNS security (15 min)
12. Ransomware resilience, backups and recovery (15 min)
13. Threat detection and security logging (14 min)
14. Encryption and key management in practice (14 min)
15. Supply chain security: SBOMs, signing and provenance (13 min)
16. Cloud security incident playbooks (13 min)
17. Investigating security alerts and failed controls (16 min)
18. Web and API attacks every operator should know (13 min)
19. SAST, DAST and dependency scanning gates (13 min)
20. Cloud posture and misconfiguration management (12 min)
21. Security champions and secure code review (12 min)
22. Incident forensics and evidence handling (13 min)
23. Audit readiness and continuous compliance evidence (12 min)
Cheat sheet

Lesson 1. The DevSecOps mindset and threat modeling

15 minutes. Move security into every stage of delivery, learn the vocabulary of risk, and use a simple threat model to find problems before attackers do.

A team ships a feature every day. Once a quarter a security group reviews the system, writes a long list of findings, and the release slips by a month. Nobody is happy, and the next quarter looks the same. DevSecOps replaces that pattern. Security checks run automatically in the same pipeline as tests, and engineers learn to think like an attacker while they design, not after the code is live.

What you will learn

- What "shift left" means and why it is not the whole story
- Core terms: asset, threat, vulnerability, risk, control
- The shared responsibility model for cloud and containers
- A short threat modeling routine using STRIDE
- Least privilege and defence in depth as design habits

Shift left, and keep watching on the right

"Shift left" means catching problems early in the lifecycle: in the editor, in a pull request, in the build. A flaw found in a design review costs minutes. The same flaw found after a breach costs months. But early checks cannot see everything, because some problems only appear in production: a new vulnerability in a library you already run, a misconfigured firewall, a stolen credential. So a mature programme does both: automated checks on the left, and monitoring, patching and incident response on the right.

The vocabulary of risk

Term	Meaning	Example
Asset	Something worth protecting	Customer database, signing key
Threat	Something that could cause harm	An attacker who steals a token
Vulnerability	A weakness that a threat can use	An unpatched library, an open port
Risk	Likelihood times impact	A public bucket with personal data
Control	A measure that reduces risk	Encryption, a firewall rule, a scan

You never remove all risk. You rank it, reduce the biggest items first, and write down the ones you accept.

Shared responsibility

A provider secures the layers it runs, and you secure what you put on top. With a virtual machine you patch the operating system. With a managed database you configure access and backups but not the host. With containers you own the image contents and the settings of the workload. When something is misconfigured, it is almost always on the customer side of this line, which is why configuration scanning matters so much.

A simple threat model with STRIDE

Threat modeling is a structured conversation about what can go wrong. Draw the system as boxes and arrows: users, services, data stores, and the trust boundaries between them, such as the internet to your load balancer. Then walk each arrow and box and ask the six STRIDE questions.

Letter	Threat	Question to ask	Typical control
S	Spoofing	Can someone pretend to be another user or service?	Strong authentication, mutual TLS

MARGA by EknathaLabs
marga.eknathalabs.com

Letter	Threat	Question to ask	Typical control
T	Tampering	Can data be changed in transit or at rest?	Signatures, integrity checks, TLS
R	Repudiation	Can someone deny an action?	Audit logs with identity and time
I	Information disclosure	Can data leak?	Encryption, access control
D	Denial of service	Can the service be made unavailable?	Rate limits, autoscaling, quotas
E	Elevation of privilege	Can a user gain rights they should not have?	Least privilege, input validation

Write each finding as a ticket with an owner. A one-hour session for each significant change finds more than a large annual review.

Diagram: Where the trust boundaries are in a simple web service

Part	What it does	What goes wrong
Browser	The person using the service. Everything coming from here is untrusted input.	If a user account is stolen, the attacker acts as that user. Rate limits and strong sign-in limit the damage.
Load balancer and WAF	Terminates TLS and filters common attacks before they reach the application.	If it fails open, attacks reach the application directly. If it fails closed, users see an outage.
Application	Business logic. It checks who the caller is and what they may do.	A flaw such as injection here exposes the data store, because the application holds the database credentials.
Database	Holds the assets: customer records.	If the database is reachable from the internet or uses a shared password, one stolen secret exposes everything.
Secrets manager	Hands short-lived credentials to the application at start-up.	If it is unavailable, new instances cannot start. If it is over-permissive, any workload can read every secret.

Flow: Browser to Load balancer and WAF / Load balancer and WAF to Application / Application to Database / Application to Secrets manager

MARGA by EknathaLabs
marga.eknathalabs.com

Least privilege and defence in depth

Least privilege means each person, service and pipeline gets only the permissions it needs, for only as long as it needs them. **Defence in depth** means no single control is trusted alone: if the firewall fails, authentication still holds, and if that fails, the data is still encrypted. Together these two habits shrink the blast radius of any mistake, which is the same idea reliability engineers use for failures.

Incident: the review that found the hole too late

> **Incident:** *A reporting service was designed, built and demoed in six weeks. In the pre-release security review, an engineer asks how the service reaches the customer database. The answer: through a shared administrator login that is stored in the code.*

A short threat model at design time would have drawn the arrow from the application to the database and asked the Spoofing and Elevation questions. The review now forces a rewrite in the last week. Compare the cost of the two paths:

```
$ grep -rn "DB_ADMIN_PASSWORD" src/ | head -3
src/report/db.py:12:DB_ADMIN_PASSWORD = "Summer2024!"
src/report/settings.py:4:DB_ADMIN_PASSWORD = os.getenv("DB_ADMIN_PASSWORD", "Summer2024!")
```

The fix is a database role limited to the reports it needs, with its password delivered at start-up. Doing that in the design review takes an hour. Doing it a week before release takes a week and delays the launch.

Common mistakes

- Treating security as a gate at the end instead of a check in every pull request
- Scanning everything, then ignoring the results because there are thousands of findings
- Giving the pipeline an administrator account because it is easier
- Writing a threat model once and never updating it when the design changes

Check yourself

What does "shift left" mean?

Answer: Running security checks earlier in the lifecycle, such as in the editor, pull request and build

Which STRIDE category describes someone pretending to be another user?

Answer: Spoofing

Under the shared responsibility model, who is normally responsible for the contents of a container image you deploy?

Answer: You, the customer

What is the best description of least privilege?

Answer: Each identity gets only the permissions it needs, for only as long as it needs them

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Secrets management

15 minutes. Keep passwords, tokens and keys out of code and logs, issue short-lived credentials, rotate on a schedule, and know exactly what to do when one leaks.

An engineer pastes a cloud access key into a test script, pushes it to a repository, and moves on. Within minutes automated scanners that watch public code find the key and start launching servers to mine cryptocurrency. The bill arrives before the engineer notices. Leaked credentials are one of the most common ways that cloud accounts are breached, and the fix is a set of habits rather than a single tool.

What you will learn

- What counts as a secret and where secrets leak
- How to inject secrets at run time instead of building them in
- Short-lived credentials and identity-based access
- Secret scanning in the pipeline
- The response when a secret is exposed

What counts as a secret

Passwords, API tokens, private keys, certificates, database connection strings, signing keys, and session cookies. A good test: if someone outside the team obtained this value, could they act as you? If yes, treat it as a secret.

Where secrets leak

- Source code and configuration files committed to a repository, including old history
- Container images, because an `ENV` or `COPY` line stores the value in a layer for good
- Build logs, when a script prints variables or runs with shell tracing on
- Chat messages, tickets and documentation pages
- Crash reports and debug endpoints that dump the environment

Deleting a file in a later commit does not remove the secret from history. Anyone who cloned the repository earlier still has it.

Inject at run time

The application should receive secrets when it starts, from a store that controls and records access, rather than carry them inside the artifact.

Method	Good for	Watch out for
Secrets manager (cloud or Vault)	Central control, audit, rotation	The permission to read it must be narrow
Orchestrator secrets	Simple workloads in a cluster	Kubernetes Secrets are base64 text, not encryption by default
Environment variables	Small setups	Visible in process listings and crash dumps
Mounted files	Certificates and keys	File permissions and tmpfs

Prefer short-lived, identity-based credentials

A long-lived key that sits in a variable is a standing risk. Better options exist:

- **Workload identity**: a pod, function or server proves who it is to the cloud provider and receives temporary credentials that expire in an hour or less. No key is stored at all.

MARGA by EknathaLabs
marga.eknathalabs.com

- **OIDC for pipelines**: the CI system presents a signed token for the job, and the cloud account exchanges it for temporary access limited to that repository and branch.
- **Dynamic secrets**: a secrets service creates a database user on demand with a short lease and deletes it afterward.

When you cannot avoid a long-lived secret, rotate it on a schedule and make rotation routine, so that rotating after an incident is not a first-time event.

Scanning for leaks

Add secret scanning in three places: a local pre-commit hook for fast feedback, a pull request check, and a periodic scan of full history and of container images. Tune the rules to reduce false positives, and make a finding block the merge, because a warning that nobody reads protects nothing.

When a secret leaks

1. **Revoke or rotate first.** Assume it was copied. Rotation matters more than removal from the repository.
2. **Check the audit logs** for use of the credential between the leak and the rotation.
3. **Remove it from where it leaked,** including history if feasible, but treat that as cleanup, not as the fix.
4. **Find the cause** and add a control, such as a scanner rule or a move to workload identity.

> **Warning:** *Rewriting repository history does not make a leaked key safe. Rotate the key before anything else.*

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Secrets management fits together

Part	What it does	What goes wrong
What counts as a secret	Passwords, API tokens, private keys, certificates, database connection strings, signing keys, and session cookies.	Giving every application permission to read every secret.
Where secrets leak	Source code and configuration files committed to a repository, including old history.	Putting secrets in ENV lines, build arguments or committed .env files.
Inject at run time	The application should receive secrets when it starts, from a store that controls and records access, rather than carry them inside the artifact.	Sharing one credential across many services so that it can never be rotated.
Prefer short-lived, identity-based	A long-lived key that sits in a variable is a standing risk.	When you cannot avoid a long-lived secret, rotate it on a schedule and make rotation routine, so that rotating after an incident is not a...
Scanning for leaks	Add secret scanning in three places: a local pre-commit hook for fast feedback, a pull request check, and a periodic scan of full history and of container images.	Check the Common mistakes list below this diagram for what usually goes wrong here.
When a secret leaks	Revoke or rotate first. Assume it was copied.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: What counts as a secret to Where secrets leak / Where secrets leak to Inject at run time / Inject at run time to Prefer short-lived, identity-based / Prefer short-lived, identity-based to Scanning for leaks / Scanning for leaks to When a secret leaks

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: the key in the test script

> **Incident:** *At 03:10 the cloud billing alert fires: 140 new large instances in a region the company has never used. The audit log shows they were launched with a deploy key that an engineer had pasted into a test script two days earlier.*

The first move is to make the key useless, then look at what it did.

```
$ aws iam update-access-key --user-name deploy --access-key-id AKIAEXAMPLEKEY --status Inactive
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=AccessKeyId,AttributeValue=AKIAEXA \
    MPLEKEY --max-results 5
```

After that, terminate the instances, check for new users and roles the key created, and move the pipeline to short-lived credentials so no static key exists at all.

Common mistakes

- Putting secrets in `ENV` lines, build arguments or committed `.env` files
- Sharing one credential across many services so that it can never be rotated
- Echoing variables in scripts and shell traces
- Giving every application permission to read every secret

Check yourself

A secret was committed and then deleted in the next commit. What should you do?

Answer: Treat it as exposed: rotate it, check audit logs, then clean up the history

Which option avoids storing a long-lived cloud key in the pipeline?

Answer: OIDC federation, where the job exchanges a signed token for temporary credentials

Why is deleting a secret in a later Dockerfile line not enough?

Answer: Earlier layers are kept in the image and can be extracted

What is the best first response to a leaked token?

Answer: Revoke or rotate it immediately

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Securing the delivery pipeline and supply chain

18 minutes. Add dependency, code and image scanning to CI, produce an inventory of what you ship, sign artifacts, and protect the pipeline itself as a target.

A modern application is mostly other people's code. A typical service pulls in hundreds of libraries, each with its own dependencies, plus a base image, a build tool and a CI runner. An attacker does not need to break your code if they can slip something into any of those. The pipeline is also powerful: it holds credentials and can deploy to production. Securing delivery means checking what goes in, proving what comes out, and hardening the machinery in between.

What you will learn

- The kinds of automated scans and what each finds
- Software bills of materials and artifact signing
- How to make scans useful rather than noisy
- Protecting the pipeline: runners, permissions and branches
- A way to describe supply chain maturity

The scanning toolbox

Check	Short name	Looks at	Finds	When it runs
Static analysis	SAST	Your source code	Injection patterns, unsafe functions	Pull request
Dependency analysis	SCA	Libraries and versions	Known vulnerabilities, risky licences	Pull request and nightly
Secret scanning		Code and history	Leaked keys and tokens	Pre-commit and pull request
Infrastructure as code scanning	IaC	Terraform, manifests	Open firewalls, public buckets	Pull request
Image scanning		Container image	Vulnerable OS and library packages	Build and registry
Dynamic testing	DAST	A running test copy	Runtime issues such as missing headers	After deploy to test

No single tool covers everything. Pick one per category and wire it into the same place engineers already look, the pull request.

```
$ trivy fs --severity HIGH,CRITICAL --exit-code 1 .
$ trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 registry.example.com/shop-ap \
    i:1.8.2
```

Make scanning useful

Thousands of findings are the same as none, because people stop reading. Four habits help:

- **Gate on severity and fixability.** Fail the build on critical and high findings that have a fix available. Track the rest as tickets.
- **Allow an expiring exception.** A suppressed finding needs an owner, a reason and a date, so exceptions do not become permanent.
- **Scan on a schedule, too.** A library that was clean at build time can have a new advisory next week.
- **Measure time to fix,** not the number of findings.

MARGA by EknathaLabs marga.eknathalabs.com

Know what you ship: SBOM

A **software bill of materials** (SBOM) lists every component in an artifact, with versions. When a new vulnerability in a popular library is announced, the question "are we affected?" should take a query, not a week. Generate an SBOM at build time, store it next to the artifact, and index it so that you can search across all services.

Prove where it came from: signing and provenance

Signing an image with a tool such as Cosign ties it to an identity. **Provenance** records how it was built: which source, which builder, which inputs. The deployment side then verifies the signature and refuses anything unsigned. The SLSA framework describes levels of assurance, from simple scripted builds to hermetic, tamper-resistant builds with verified provenance.

Protect the pipeline itself

- **Least privilege for jobs.** A test job needs no deploy rights. Give each job its own narrow, short-lived identity.
- **Isolated, ephemeral runners.** Build on fresh machines that are destroyed after the job, so one job cannot leave something for the next.
- **Protected branches and reviews.** Require review and passing checks before merge. Limit who can change the pipeline definition, because that file is code with production access.
- **Pin third-party actions and plugins** to exact versions or digests. A tag can be moved by its owner.
- **Treat pull requests from forks as untrusted.** Do not expose secrets to them.

> **Danger:** *Never run untrusted code, such as a pull request from an outside fork, on a runner that holds production credentials.*

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Securing the delivery pipeline and supply chain fits together

Part	What it does	What goes wrong
The scanning toolbox	No single tool covers everything. Pick one per category and wire it into the same place engineers already look, the pull request.	Running the whole pipeline with one powerful shared credential.
Make scanning useful	Thousands of findings are the same as none, because people stop reading.	Trusting a dependency because it is popular.
Know what you ship: SBOM	A software bill of materials (SBOM) lists every component in an artifact, with versions.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Prove where it came from: signing	Signing an image with a tool such as Cosign ties it to an identity.	Using the latest tag for base images and tools.
Protect the pipeline itself	Least privilege for jobs. A test job needs no deploy rights.	Build on fresh machines that are destroyed after the job, so one job cannot leave something for the next.

Flow: The scanning toolbox to Make scanning useful / Make scanning useful to Know what you ship: SBOM / Know what you ship: SBOM to Prove where it came from: signing / Prove where it came from: signing to Protect the pipeline itself

Incident: a poisoned dependency

> **Incident:** *A build starts to fail on a new check. A routine update pulled in version 4.1.2 of a small, popular library. Its install script now sends the contents of environment variables to a remote host.*

Because the team keeps an SBOM for each release, the first question is easy to answer:

MARGA by EknathaLabs marga.eknathalabs.com

```
$ grep -l '"name": "tiny-utils"' sboms/*.json
sboms/billing-2.8.0.json
sboms/checkout-5.2.1.json
```

Two services are affected. Pin the previous version, rotate every secret that was available to those builds, rebuild from a clean runner, and add a rule that new versions of dependencies wait a few days before they are accepted.

Common mistakes

- Using the latest tag for base images and tools
- Running the whole pipeline with one powerful shared credential
- Turning scanners on in report-only mode forever
- Trusting a dependency because it is popular

Check yourself

What is the main purpose of an SBOM?

Answer: To list every component in an artifact so you can answer "are we affected?" when a vulnerability is announced

Which scan checks Terraform files for an open firewall or public bucket?

Answer: Infrastructure as code scanning

Why pin a base image by digest?

Answer: The content cannot change under the same reference, so builds are reproducible

What is a sensible policy for scan failures?

Answer: Fail on critical and high findings that have a fix, and track the rest with owners and dates

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Container and image security

15 minutes. Build minimal images, run as a non-root user with dropped capabilities, scan and sign, and use runtime controls to limit what a compromised container can do.

A container is a process with a fence around it, and the fence is only as strong as its settings. A default container often runs as root, has more Linux capabilities than it needs, can write anywhere in its filesystem, and ships with a shell and package manager an attacker can use. Hardening removes those conveniences one by one so that a bug in the application has less to work with.

What you will learn

- Choosing and maintaining a minimal base image
- Running with a non-root user and fewer privileges
- Runtime restrictions: read-only filesystems, capabilities, seccomp
- Image scanning and registry policy
- What not to do: privileged mode and the Docker socket

Start with less

Every package in an image is something to patch and something an attacker can use. Prefer a slim or distroless base with no shell, build with a multi-stage Dockerfile so compilers stay behind, and rebuild regularly to pick up fixes. Pin the version, and use a `.dockerignore` file so that credentials and local files never enter the build context.

```
FROM golang:1.22 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /out/server .

FROM gcr.io/distroless/static:nonroot
COPY --from=build /out/server /server
USER 65532:65532
ENTRYPOINT ["/server"]
```

Do not run as root

If the process runs as root inside the container and an escape bug exists, the attacker may be root on the host. Set `USER` to an unprivileged ID and make sure files the app must write are owned by that user. In Kubernetes enforce it with `runAsNonRoot: true`.

Runtime restrictions

```
docker run --rm \
  --read-only --tmpfs /tmp \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  --pids-limit 200 --memory 256m \
  registry.example.com/shop-api:1.8.2
```

Setting	What it does
Read-only root filesystem	An attacker cannot drop tools or change binaries
Drop all capabilities	Removes special powers such as raw network access
no-new-privileges	Blocks setuid programs from gaining rights
Seccomp profile	Limits which system calls the process may make
Memory and process limits	Stop one container from starving the host

MARGA by EknathaLabs marga.eknathalabs.com

Scanning and registry policy

Scan in the build and again in the registry, since new advisories appear daily. Keep a private registry, allow only approved base images, sign images, and have the cluster verify signatures at admission. Remove old tags that nobody should deploy.

> **Danger:** *Never use* `--privileged` *or mount the Docker socket into an application container. Both give the container control of the host.*

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Container and image security fits together

Part	What it does	What goes wrong
Start with less	Every package in an image is something to patch and something an attacker can use.	Adding a shell and debugging tools to the production image.
Do not run as root	If the process runs as root inside the container and an escape bug exists, the attacker may be root on the host.	Running as root because it was the default.
Runtime restrictions	.	Scanning only at build time and never again.
Scanning and registry policy	Scan in the build and again in the registry, since new advisories appear daily.	Passing secrets with build arguments or ENV.

Flow: Start with less to Do not run as root / Do not run as root to Runtime restrictions / Runtime restrictions to Scanning and registry policy

Incident: the shell in the web container

> **Incident:** *A runtime alert reports an interactive shell started inside a production web container, followed by a download of an unknown binary.*

The container ran as root with a writable filesystem and a package manager, which made the attacker's work easy. Check what the container was allowed to do, then replace it with a hardened one.

```
$ docker inspect web --format 'user={{.Config.User}} readonly={{.HostConfig.ReadonlyRootfs}} caps= \
    {{.HostConfig.CapDrop}}'
user= readonly=false caps=[]
```

The empty user means root, and nothing was dropped. The fix is a distroless image, a non-root user, a read-only filesystem and dropped capabilities, so that the same bug leads nowhere.

Common mistakes

- Running as root because it was the default
- Adding a shell and debugging tools to the production image
- Passing secrets with build arguments or `ENV`
- Scanning only at build time and never again

Check yourself

Which setting prevents an attacker from modifying binaries in a running container?

Answer: A read-only root filesystem

Why is mounting the Docker socket into a container dangerous?

Answer: Anyone who can use the socket can start privileged containers and take over the host

What is a benefit of a distroless base image?

Answer: It contains little beyond the application, so there is less to patch and less for an attacker to use

MARGA by EknathaLabs marga.eknathalabs.com

What does --cap-drop ALL do?

Answer: Removes the special Linux capabilities, so the process has only ordinary user powers

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Kubernetes security

18 minutes. Control who can do what with RBAC, restrict pod behaviour with security standards and policies, isolate traffic with network policies, and protect secrets and the control plane.

A Kubernetes cluster is a shared system with a powerful API. By default, pods can talk to every other pod, a pod can often reach the API server with a token, and nothing stops a deployment from asking for root or for the host network. Securing a cluster means narrowing each of those defaults, one layer at a time.

What you will learn

- RBAC and service accounts
- Pod security standards and admission policies
- Network policies and default deny
- Protecting secrets and the control plane
- Runtime detection and audit logs

Identity: RBAC and service accounts

Role-based access control decides which verbs (get, list, create, delete) a subject may use on which resources in which namespace. Grant roles to groups, not individuals, prefer namespaced `Role` over `ClusterRole`, and avoid wildcards.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deploy-reader
  namespace: shop
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch"]
```

Every pod runs as a service account. Create a dedicated one per workload, and set `automountServiceAccountToken: false` for pods that never call the API. Check access with `kubectl auth can-i`:

```
$ kubectl auth can-i delete pods -n shop --as system:serviceaccount:shop:web
no
```

Pod security

The built-in Pod Security Standards define three levels: privileged, baseline and restricted. Enforce at least baseline everywhere, and restricted for application namespaces, by labelling the namespace. The restricted level requires non-root, no privilege escalation, dropped capabilities and a safe seccomp profile.

```
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
  seccompProfile:
    type: RuntimeDefault
```

For rules beyond the standards, such as "images must come from our registry" or "every workload needs resource limits", use an admission policy engine such as Kyverno or OPA Gatekeeper, or the built-in validating admission policies. Start in audit mode, fix the findings, and then enforce.

Network policies

A NetworkPolicy is a firewall rule for pods. Start with a default deny for ingress in each namespace, then allow only the flows you intend. This needs a network plugin that enforces policies.

```
apiVersion: networking.k8s.io/v1
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
kind: NetworkPolicy
metadata: { name: default-deny, namespace: shop }
spec:
  podSelector: {}
  policyTypes: ["Ingress"]
```

Diagram: What an attacker can reach after taking over one pod

Part	What it does	What goes wrong
Compromised pod	A web pod that an attacker has gained a shell in.	The starting point. Its permissions decide how far the attacker gets.
API server	The control plane entry point. It accepts calls with the pod service account token.	If the token has broad rights, the attacker can read secrets or start new pods. Narrow RBAC and no auto-mounted token stop this.
Database pod	Holds customer data and listens inside the cluster.	With no network policy, any pod can connect. A default deny policy blocks the connection.
Worker node	The host that runs the pod.	A privileged pod or a mounted host path can reach the node and every other pod on it.

Flow: Compromised pod to API server / Compromised pod to Database pod / Compromised pod to Worker node

Secrets and the control plane

- Turn on **encryption at rest** for Secrets in etcd, and restrict who can read them. A Secret is base64 text, not protection.
- Prefer an external secrets store or a CSI driver that mounts secrets on demand.
- Never expose the API server or the kubelet to the whole internet. Restrict by network and require authentication.
- Keep Kubernetes and node images updated, and enable audit logging for the API.

> **Warning:** *Granting a user the right to create pods in a namespace can be equal to granting access to every secret in it. Review such roles with care.*

Runtime detection

Admission controls stop bad configuration. They do not see an attacker who uses a legitimate pod. A runtime tool, such as Falco or eBPF-based agents, watches system calls and alerts on events such as a shell starting in a container, a write to a sensitive path, or an unexpected outbound connection.

Incident: one pod, whole namespace

> **Incident:** *A vulnerable web pod is exploited. Within minutes the attacker lists secrets in the namespace and connects to the database pod directly.*

Two default behaviours made it easy: the pod had a service account token with broad rights, and nothing blocked traffic between pods.

```
$ kubectl auth can-i list secrets -n shop --as system:serviceaccount:shop:web
yes
$ kubectl get networkpolicy -n shop
No resources found in shop namespace.
```

After containment, give each workload its own narrow service account, stop mounting tokens where they are not needed, add a default deny policy with explicit allows, and enforce the restricted pod security level.

Common mistakes

- Binding `cluster-admin` to a service account or to every developer

MARGA by EknathaLabs marga.eknathalabs.com

- Leaving the default service account token mounted in all pods
- Running with no network policies, so that a flat network exists
- Enforcing a policy engine before auditing what it would block

Check yourself

What does a default deny NetworkPolicy do?

Answer: Blocks all ingress to the selected pods until other policies allow specific traffic

Which setting prevents a pod from getting extra privileges through setuid programs?

Answer: allowPrivilegeEscalation: false

Why set automountServiceAccountToken to false for many pods?

Answer: A pod that never calls the API does not need a credential that an attacker could steal

Are Kubernetes Secrets encrypted by default in etcd?

Answer: No, they are base64 encoded unless you enable encryption at rest

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Cloud security, identity and infrastructure as code

18 minutes. Design least-privilege identity, protect data with encryption and private networking, turn on audit logging, and check Terraform and cloud accounts for misconfiguration.

In the cloud, the control panel is the new perimeter. A single over-broad role or a public storage bucket can expose more data than any network attack. And because infrastructure is created by code, one mistaken template can reproduce the same weakness in fifty accounts. Cloud security is mostly identity, configuration and visibility.

What you will learn

- Designing least-privilege IAM policies
- Account structure and guardrails
- Encryption and private connectivity
- Logging and detection
- Scanning infrastructure as code and live accounts

Identity is the perimeter

Use roles that are assumed temporarily rather than users with permanent keys. Give people access through single sign-on with multi-factor authentication, and give workloads their own roles. Build policies from what is needed, not from what is convenient.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::shop-reports/monthly/*"
  }]
}
```

This policy allows one action on one prefix. A wildcard policy such as `"Action": "*", "Resource": "*"` lets whoever holds it do anything in the account. Use the access advisor and access analyzer tools to see which permissions are actually used and to trim the rest.

Accounts, guardrails and separation

Split workloads into separate accounts or subscriptions for production, testing and shared tooling. Apply organisation-wide guardrails that no account administrator can lift, such as blocking public storage, forbidding unapproved regions, and protecting the logging configuration. A blast radius that ends at an account boundary is easy to reason about.

Diagram: A request for a private file in a well-configured cloud account

Part	What it does	What goes wrong
Signed-in user	Authenticated with single sign-on and multi-factor authentication.	If sign-in is weak, an attacker becomes a legitimate user. Multi-factor authentication limits that.
Workload role	The temporary identity of the application. It can read one bucket prefix.	An over-broad role turns any application bug into a data breach.
Encryption key	Encrypts the data. Use of the key is logged and limited by key policy.	If the key policy is too open, encrypted data gives no real protection.
Private bucket	Stores reports. Block public access is on.	A public setting exposes every object to the internet.
Audit log	Records every API call, in a separate protected account.	If logs can be deleted by the account owner, an attacker removes the evidence.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Signed-in user to Workload role / Workload role to Private bucket / Encryption key to Private bucket / Private bucket to Audit log

Protect the data

- **Encrypt at rest and in transit.** Use managed keys, or customer-managed keys when you need separation of duties.
- **Keep data stores private.** Place databases in private subnets and reach cloud services through private endpoints, not the public internet.
- **Block public access at the account level** for object storage, so one wrong bucket setting cannot expose it.
- **Back up and protect backups** with a different account or immutable storage, so ransomware cannot delete both.

Log, alert, respond

Enable the cloud's audit trail in all regions and send it to a protected account. Alert on high-risk events: root account use, a new access key, a change to logging, a policy that grants public access, or a role with broad rights. Logs that nobody reads do not protect you, and logs that an attacker can erase do not help later.

Scan before and after deploy

Scan Terraform, Kubernetes manifests and templates in the pull request, with tools such as Checkov, Trivy or tfsec, so that an open security group never reaches production. Then scan live accounts with a posture management tool, because people also change things in the console. Detect drift and either reconcile it or turn the console into read-only for most people.

> **Warning:** *Do not put secrets in Terraform variables or state without protection. State files often contain passwords in plain text, so encrypt the backend and restrict access.*

Incident: the wildcard role

> **Incident:** *An application with a server-side request forgery bug lets an attacker call the instance metadata service and obtain the role credentials. The role allows every action on every bucket.*

With a narrow role the same bug would have exposed one prefix. Check what the role really allows:

```
$ aws iam get-role-policy --role-name app-role --policy-name inline
{"Statement": [{"Effect": "Allow", "Action": "s3:*", "Resource": "*"}]}
```

Replace it with the specific actions and prefixes the application uses, require the newer metadata service version that resists this attack, and alert on use of the credentials from outside the instance.

Common mistakes

- Using the root account for daily work
- Granting wide wildcard permissions "to get it working" and never narrowing them
- Storing logs in the same account that an attacker will control
- Assuming the provider secures your configuration

Check yourself

What is the best way to give an application access to a bucket?

Answer: Attach a role with a narrow policy to the workload and use temporary credentials

Why send audit logs to a separate protected account?

Answer: So that someone who takes over a workload account cannot erase the evidence

MARGA by EknathaLabs marga.eknathalabs.com

What does scanning Terraform in the pull request achieve?

Answer: It finds problems such as open firewalls before they are created

Which is a risk of Terraform state files?

Answer: They can contain secret values in plain text, so the backend must be encrypted and access limited

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Vulnerability management and security incidents

15 minutes. Prioritise findings by real risk, patch within agreed times, detect attacks from logs and signals, and run a security incident with clear roles and evidence.

Preventive controls will not stop everything. A new vulnerability will be announced on a Friday, a credential will be phished, a dependency will be hijacked. The measure of a security programme is how fast it finds out and how well it responds. This lesson covers the loop that follows the build: managing vulnerabilities, detecting trouble, and handling an incident.

What you will learn

- Prioritising vulnerabilities by risk, not by score alone
- Patch service levels and how to meet them
- What signals reveal an attack
- The phases of a security incident
- How to preserve evidence and communicate

Prioritise by real risk

A CVSS score describes how severe a flaw is in general. Real risk depends on your situation. Ask four questions:

1. Is the vulnerable component actually used and reachable, or only installed?
2. Is the service exposed to the internet?
3. Is an exploit known to be in active use? (Public catalogues of exploited vulnerabilities exist for exactly this purpose.)
4. What data or privileges are at stake?

A medium flaw on an exposed login service with a public exploit outranks a critical flaw in a library that the code never calls.

Service levels for fixing

Agree and publish target times, for example: critical and actively exploited within 48 hours, high within 14 days, medium within 60 days. Track the percent that meet the target and the median time to fix. Automate updates with a bot that opens pull requests for new versions, so that routine patches are small and frequent instead of rare and risky.

Severity	Example target	Typical action
Critical, exploited	48 hours	Patch or mitigate now, outside the normal cycle
High	14 days	Next sprint, with owner
Medium	60 days	Planned maintenance
Low	Best effort	Fix with other changes

Detection

You need telemetry to see an attack: authentication logs, cloud audit logs, DNS and network flow logs, container runtime events, and application errors. Good detections are specific:

- A login from a new country followed by a change of multi-factor settings
- A role assumed from an unusual address
- A shell started inside a production container

MARGA by EknathaLabs marga.eknathalabs.com

- Large outbound transfers from a database host
- A sudden jump in denied API calls, which suggests probing

Route alerts to the same on-call system as reliability alerts, with runbooks that say what to check first.

The incident phases

1. **Prepare.** Roles, contact lists, runbooks and practice exercises, before you need them.
2. **Detect and triage.** Is this real? How far has it spread?
3. **Contain.** Stop the damage: disable the credential, isolate the host, block the address. Prefer isolating over deleting, so you keep evidence.
4. **Eradicate and recover.** Remove the attacker's access, rebuild from known good images, rotate secrets, then restore service.
5. **Learn.** A blameless review that produces changes to controls.

Evidence and communication

Keep a timeline of actions with times and who did them. Write down commands that you ran. Limit who is in the incident channel, and agree who speaks to customers, regulators and the press. Some regions require notifying authorities within a fixed time after a personal data breach, so involve legal staff early.

> **Warning:** *Do not log in to a compromised machine with the credentials of an administrator whose account may be the target. Use a clean workstation and a fresh, narrowly scoped account.*

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Vulnerability management and security incidents fits together

Part	What it does	What goes wrong
Prioritise by real risk	A CVSS score describes how severe a flaw is in general.	Chasing the score and ignoring exposure.
Service levels for fixing	Agree and publish target times, for example: critical and actively exploited within 48 hours, high within 14 days, medium within 60 days.	Treating patching as an annual project instead of a weekly habit.
Detection	You need telemetry to see an attack: authentication logs, cloud audit logs, DNS and network flow logs, container runtime events, and application errors.	Collecting logs but writing no alerts.
The incident phases	Prepare. Roles, contact lists, runbooks and practice exercises, before you need them.	Wiping machines before capturing evidence.
Evidence and communication	Keep a timeline of actions with times and who did them.	> Warning: Do not log in to a compromised machine with the credentials of an administrator whose account may be the target.

Flow: Prioritise by real risk to Service levels for fixing / Service levels for fixing to Detection / Detection to The incident phases / The incident phases to Evidence and communication

Incident: a Friday advisory

> **Incident:** *On Friday afternoon a critical flaw in a widely used logging library is announced, and exploit code is already circulating. Leadership asks: are we affected?*

With stored SBOMs the answer takes minutes, and with an exposure list the order of work is clear.

```
$ grep -l '"name": "loglib"' sboms/*.json | wc -l
7
```

MARGA by EknathaLabs | marga.eknathalabs.com

```
$ ./exposed.sh --services $(grep -l loglib sboms/*.json | xargs -n1 basename | sed 's/-.*//')
checkout   internet-facing   version 2.14.1  patch available
batch-3    internal          version 2.14.1  patch available
```

Patch or mitigate the internet-facing service first, then the others. Watch logs for exploit attempts since the advisory, and record the timeline for the review.

Common mistakes

- Chasing the score and ignoring exposure
- Treating patching as an annual project instead of a weekly habit
- Collecting logs but writing no alerts
- Wiping machines before capturing evidence

Check yourself

Which finding should usually be fixed first?

Answer: A high flaw on an internet-facing service with a public exploit in active use

What is the first step in containing a compromised host?

Answer: Isolate it from the network and preserve evidence

Why automate dependency update pull requests?

Answer: Frequent small updates are easier to test and keep fix times short

What makes a detection useful?

Answer: It is specific, has a runbook, and goes to someone who can act

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Identity, authentication and zero trust

16 minutes. Understand how sign-in, tokens and multi-factor authentication work, apply OAuth and OIDC safely, authenticate services with mutual TLS, and design with zero trust instead of a trusted network.

For years a company network was treated like a castle: strong walls outside, and anyone inside was trusted. Remote work, cloud services and stolen credentials broke that model. An attacker with one valid login on the inside could reach almost everything. Zero trust replaces "inside means safe" with a simple rule: verify every request, from every user and every service, every time, using identity.

What you will learn

- The difference between authentication and authorization
- How tokens, OAuth and OIDC fit together
- Multi-factor authentication and phishing-resistant methods
- Service-to-service identity and mutual TLS
- The principles of zero trust and how to start

Authentication and authorization

Authentication answers "who are you?". **Authorization** answers "what may you do?". Mixing them up causes many flaws: an application that checks the login but not the permission lets any signed-in user read any record by changing an ID in the URL.

Tokens, OAuth and OIDC

Applications rarely handle passwords themselves. They rely on an identity provider and receive tokens.

Term	Purpose	Example
OAuth 2.0	Delegated authorization: let an app act on your behalf with limited scope	An app gets read-only access to your calendar
OpenID Connect (OIDC)	Authentication layer on top of OAuth: tells the app who the user is	"Sign in with your company account"
Access token	Short-lived proof of permission, sent with API calls	A signed JWT valid for 10 minutes
Refresh token	Longer-lived credential used to get new access tokens	Stored securely, rotated on use

Good habits: keep access tokens short-lived, validate the signature, issuer, audience and expiry on every request, request the smallest scope, and never put tokens in URLs or logs.

```
$ echo "$TOKEN" | cut -d. -f2 | base64 -d 2>/dev/null | jq '{iss, aud, exp, scope}'
{
  "iss": "https://login.example.com",
  "aud": "orders-api",
  "exp": 1767225600,
  "scope": "orders:read"
}
```

A JWT is signed, not encrypted: anyone can read the payload, so never place secrets in it.

Multi-factor authentication

A password alone is weak because it can be guessed, reused or phished. A second factor adds something you have or are. Not all factors are equal.

Method	Strength	Weakness
SMS code	Better than nothing	Can be intercepted or SIM swapped

MARGA by EknathaLabs
marga.eknathalabs.com

Method	Strength	Weakness
Authenticator app code	Good	A fake site can capture and replay the code
Push approval	Convenient	Users may approve a prompt they did not start
Hardware key or passkey	Strongest	Needs enrolment and recovery planning

Passkeys and hardware keys are bound to the real website address, so a look-alike phishing site cannot use them. Require them for administrators and for access to production.

Service identity and mutual TLS

Services need identities too. With ordinary TLS only the client checks the server. With **mutual TLS** (mTLS) both sides present certificates, so a service can know exactly which caller it is talking to. A service mesh or a workload identity system such as SPIFFE can issue short-lived certificates automatically, so nobody manages keys by hand.

Diagram: Zero trust: every request is checked

Part	What it does	What goes wrong
User and device	Signs in with a phishing-resistant method from a managed device.	A stolen password with no second factor lets an attacker in. Device checks and keys reduce that.
Identity-aware proxy	Verifies who the user is, what device they use and which application they may reach, on every request.	If it is bypassed by a direct path to the app, all checks are skipped. Close the direct path.
Application	Checks permissions again for the specific action and data requested.	If it trusts the proxy blindly, one flaw in the proxy exposes everything. Authorize in the app too.
Backend service	Accepts calls only from services that present a valid certificate (mTLS).	With plain network trust, any compromised host can call it.
Policy and logs	Central rules decide access, and every decision is logged.	If logs are missing, you cannot investigate. If rules are loose, the model is zero trust in name only.

Flow: User and device to Identity-aware proxy / Identity-aware proxy to Application / Application to Backend service / Policy and logs to Identity-aware proxy / Policy and logs to Application

Zero trust in practice

1. **Know your identities and devices.** Inventory users, services and the machines they use.
2. **Authenticate strongly and often.** Prefer phishing-resistant factors, short sessions and re-checks for sensitive actions.
3. **Grant least privilege, just in time.** Elevated access is requested, approved and expires.
4. **Segment the network** and treat the internal network as untrusted.
5. **Log and watch** every access decision.

Start with the highest-risk systems: administrator access and production. Moving everything at once is not realistic.

Incident: the forgotten service account

Incident: *An audit finds a service account with administrator rights, a password that has never changed, and no owner. Its password appears in an old wiki page.*

```
$ aws iam list-users --query 'Users[].UserName' --output text
alice bob svc-backup-old
$ aws iam get-user --user-name svc-backup-old --query 'User.PasswordLastUsed'
```

MARGA by EknathaLabs marga.eknathalabs.com

```
"2023-02-11T09:41:00+00:00"
```

Disable it, check the logs for recent use, find what depended on it, and replace the dependency with a workload role that has narrow permissions and short-lived credentials. Add an owner and an expiry date to every service identity.

Common mistakes

- Checking that a user is logged in, but not that they may access that record
- Long-lived tokens and tokens in URLs
- Trusting traffic because it comes from the internal network
- Shared accounts that make audit logs meaningless

Check yourself

What is the difference between OAuth 2.0 and OIDC?

Answer: OAuth handles delegated authorization, and OIDC adds authentication so an app knows who the user is

Why are hardware keys and passkeys resistant to phishing?

Answer: They are bound to the real site address, so a look-alike site cannot use them

What does mutual TLS add compared with normal TLS?

Answer: The server also verifies the client certificate, so services know who is calling

What is the core idea of zero trust?

Answer: Verify every request using identity and context, and do not trust a request just because of where it comes from

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Application security for DevOps engineers

16 minutes. Recognise the most common web and API weaknesses, understand how input validation, headers and rate limits stop them, and review code and configuration for risky patterns.

Infrastructure can be perfect and the application can still be the way in. Most breaches of internet-facing systems use a small set of well-known mistakes: unchecked input, broken access control, weak session handling and unsafe use of third-party code. DevOps engineers do not need to be penetration testers, but they run the pipelines, gateways and configuration that decide whether those mistakes get caught.

What you will learn

- The categories in the OWASP Top 10 that matter most in operations
- How injection and request forgery attacks work, in plain terms
- Security headers, cookies and TLS settings
- API protection: authentication, rate limiting and input limits
- What to look for in code review and in a scan report

The mistakes that keep appearing

Weakness	What happens	Typical fix
Broken access control	A user changes an ID and reads someone else's data	Check permission for every object on the server
Injection	Input is treated as code, for example in SQL or a shell command	Parameterised queries, no string building
Security misconfiguration	Debug pages on, default passwords, open storage	Hardened defaults, configuration scanning
Vulnerable components	An old library with a known flaw	Dependency scanning and updates
Server-side request forgery	The server fetches a URL chosen by the attacker, including internal addresses	Allow lists for outbound requests, block metadata addresses
Weak authentication	Guessable passwords, no lockout, long sessions	MFA, rate limits, short sessions
Logging failures	Attacks are not recorded or noticed	Log security events, alert on patterns

Injection in one example

```
# Unsafe: user input becomes part of the query text
query = "SELECT * FROM users WHERE name = '" + name + "'"

# Safe: the driver sends the value separately from the query
cursor.execute("SELECT * FROM users WHERE name = %s", (name,))
```

If `name` is `x' OR '1'='1`, the first version returns every user. The second treats the whole text as a value. The same idea applies to shell commands, LDAP queries and template engines.

Headers, cookies and TLS

Set these at the gateway or in the web server so every application benefits:

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header Content-Security-Policy "default-src 'self'" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
```

Session cookies should have `Secure`, `HttpOnly` and `SameSite` attributes. Redirect HTTP to HTTPS, and disable old TLS versions.

MARGA by EknathaLabs marga.eknathalabs.com

Protecting APIs

- **Authenticate every call** and authorize each object, not only the endpoint.
- **Rate limit** per user and per address, to slow brute force attempts and abuse.
- **Limit input**: maximum body size, field lengths, number of items per page.
- **Return little**: do not send fields the client does not need, and avoid detailed error messages.
- **Keep an inventory** of every API, including old versions, because forgotten endpoints are often unprotected.

Diagram: Layers that protect an API

Part	What it does	What goes wrong
Client	Any caller, including attackers. All input is untrusted.	Nothing to fix here. Assume hostile input.
Gateway and WAF	Terminates TLS, applies rate limits, blocks known attack patterns and large bodies.	If it is bypassed through a direct address, the app is exposed. If rules are too loose, attacks pass.
Authentication	Validates the token: signature, issuer, audience and expiry.	If validation is skipped or weak, anyone can forge a token.
API code	Checks object-level permission and validates input.	Missing permission checks let users read each other's data.
Database	Accessed with a least-privilege account and parameterised queries.	An admin account here turns one injection into a full breach.

Flow: Client to Gateway and WAF / Gateway and WAF to Authentication / Gateway and WAF to API code / Authentication to API code / API code to Database

Incident: the guessable invoice number

> **Incident:** *A customer notices that the invoice link ends with a number. Changing it to the next number shows another company's invoice.*

```
$ curl -s -H "Authorization: Bearer $TOKEN" https://api.example.com/invoices/10042 | jq .customer
"Acme Ltd"
$ curl -s -H "Authorization: Bearer $TOKEN" https://api.example.com/invoices/10043 | jq .customer
"Globex Corp"
```

The API checks that the caller is logged in, but not that the invoice belongs to the caller. The fix is a server-side ownership check on every request, plus an alert for one account reading many objects in sequence. Random identifiers help, but they do not replace the check.

Common mistakes

- Relying on the client or front end to enforce permissions
- Building queries and commands by joining strings
- Returning stack traces or internal details in error messages
- Leaving old API versions running without protection

Check yourself

What is the best defence against SQL injection?

Answer: Parameterised queries, where values are sent separately from the SQL text

What is broken access control?

Answer: The server fails to check that the user may access the specific data or action requested

Why is server-side request forgery dangerous in the cloud?

Answer: The attacker can make the server call internal addresses, such as the metadata service, and steal credentials

MARGA by EknathaLabs marga.eknathalabs.com

Which cookie attribute helps stop JavaScript from reading a session cookie?

Answer: HttpOnly

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Policy as code, compliance and audit evidence

15 minutes. Turn security rules into checks that run automatically, use admission and pipeline policies to prevent drift, and collect continuous evidence for audits instead of screenshots.

Security teams write policies: encrypt data, restrict public access, patch within a set time. Auditors ask for proof. When policies live in documents and proof lives in screenshots, both go out of date by the time they are read. Policy as code puts the rule in a machine-readable form that is checked on every change, and compliance as code produces the evidence automatically.

What you will learn

- What policy as code is and where it can run
- Writing simple policies with Open Policy Agent and Kyverno
- Guardrails in the cloud and in pipelines
- Frameworks such as CIS, SOC 2 and ISO 27001, in plain terms
- Collecting evidence continuously

Where policy runs

Point	What it checks	Example
Editor and pull request	Terraform and manifests before merge	No public bucket
Pipeline	Images and artifacts before release	Only signed images from our registry
Admission	Objects as they enter the cluster	Pods must run as non-root
Cloud account	Settings of live resources	Block public storage, deny unapproved regions
Runtime and audit	Drift and exceptions after the fact	A security group opened to the world

The earlier in the line, the cheaper the fix. The later controls catch what slipped through and what changed by hand.

A policy in Rego

Open Policy Agent uses a language called Rego. This rule denies Terraform plans that open SSH to the internet:

```
package terraform.security

deny[msg] {
  rc := input.resource_changes[_]
  rc.type == "aws_security_group_rule"
  rc.change.after.from_port <= 22
  rc.change.after.to_port >= 22
  rc.change.after.cidr_blocks[_] == "0.0.0.0/0"
  msg := sprintf("%s opens SSH to the world", [rc.address])
}
```

The pipeline runs the check on the plan and fails the build with a clear message.

A policy for Kubernetes

Kyverno writes policies as Kubernetes objects. This one requires every pod to run as non-root:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-non-root
spec:
  validationFailureAction: Audit
```

MARGA by EknathaLabs marga.eknathalabs.com

```
rules:
  - name: run-as-non-root
    match:
      any:
        - resources:
            kinds: ["Pod"]
    validate:
      message: "Containers must set runAsNonRoot to true."
      pattern:
        spec:
          securityContext:
            runAsNonRoot: true
```

Start in `Audit` mode to see what would be blocked. Fix the findings, then switch to `Enforce`.

Guardrails versus gates

A **gate** blocks a change until a person approves. A **guardrail** makes the unsafe action impossible or automatically corrected, with no waiting. Guardrails scale better: a service control policy that denies public storage in every account needs no reviewer.

Frameworks in plain words

- **CIS Benchmarks**: detailed lists of secure settings for operating systems, clusters and cloud accounts.
- **SOC 2**: an audit report about how a service provider handles security, availability and confidentiality.
- **ISO 27001**: a standard for running an information security management system.
- **PCI DSS** and **HIPAA**: rules for card data and health data.

They differ in detail, yet they ask for similar things: access control, encryption, logging, change management, patching and incident response. Map each rule you automate to the controls it supports once, and reuse it across frameworks.

Continuous evidence

Auditors want to see that a control worked over time. Instead of taking screenshots, store the output of your automated checks with dates: pipeline results, policy reports, access reviews and change records. Keep it in a place that cannot be edited afterward.

```
$ aws configservice describe-compliance-by-config-rule --query 'ComplianceByConfigRules[].[ConfigR \
    uleName,Compliance.ComplianceType]' --output text
s3-bucket-public-read-prohibited    COMPLIANT
encrypted-volumes                   NON_COMPLIANT
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Policy as code, compliance and audit evidence fits together

Part	What it does	What goes wrong
Where policy runs	The earlier in the line, the cheaper the fix.	Enforcing a new policy without an audit period.
A policy in Rego	Open Policy Agent uses a language called Rego.	Writing policies in documents that nobody checks.
A policy for Kubernetes	Kyverno writes policies as Kubernetes objects.	Granting exceptions with no owner or expiry.
Guardrails versus gates	A gate blocks a change until a person approves.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Frameworks in plain words	CIS Benchmarks: detailed lists of secure settings for operating systems, clusters and cloud accounts.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Continuous evidence	Auditors want to see that a control worked over time.	Collecting screenshots for audits instead of automatic evidence.

Flow: Where policy runs to A policy in Rego / A policy in Rego to A policy for Kubernetes / A policy for Kubernetes to Guardrails versus gates / Guardrails versus gates to Frameworks in plain words / Frameworks in plain words to Continuous evidence

Incident: the exception that never expired

> **Incident:** *A temporary exception to the "no public storage" rule was granted for a launch. Two years later an audit finds the bucket still public, with customer exports inside.*

An exception needs an owner, a reason and an end date, and the policy engine should remove it automatically. Look for existing exceptions regularly:

```
$ grep -rn "exception" policy/exceptions.yaml | head -3
policy/exceptions.yaml:12:  - bucket: launch-assets  owner: ""  expires: ""
```

Add the missing fields as required, fail the check when they are empty, and review exceptions every quarter.

Common mistakes

- Writing policies in documents that nobody checks
- Enforcing a new policy without an audit period
- Granting exceptions with no owner or expiry
- Collecting screenshots for audits instead of automatic evidence

Check yourself

What is policy as code?

Answer: Security rules written in a machine-readable form and checked automatically on every change

Why start a new Kubernetes policy in audit mode?

Answer: To see what would be blocked and fix it before enforcing, so teams are not surprised

What is a guardrail?

Answer: A control that makes unsafe actions impossible or corrects them automatically

What makes a good exception to a policy?

Answer: An owner, a reason and an end date, reviewed regularly

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 11. Network, edge and DNS security

15 minutes. Segment networks, control outbound traffic, protect the edge with TLS, WAF and DDoS defences, and secure DNS and certificates.

Attackers need two things: a way in and a way to move and take data out. Good network security makes each step hard. It limits what can talk to what, watches the traffic that leaves, and protects the edge where the internet meets your services. These controls still matter in the cloud, where they are expressed as security groups, route tables and managed services instead of cables and appliances.

What you will learn

- Segmentation and security groups that follow least privilege
- Controlling outbound (egress) traffic
- Protecting the edge: TLS, WAF and DDoS mitigation
- DNS security and certificate hygiene
- Private connectivity to cloud services

Segmentation

Divide the network into zones by function and risk, and allow only the flows that are needed: internet to load balancer, load balancer to application, application to database. Nothing else.

```
# Review: which security groups allow the whole internet?
aws ec2 describe-security-groups \
  --filters Name=ip-permission.cidr,Values=0.0.0.0/0 \
  --query 'SecurityGroups[].[GroupId,GroupName]' --output text
```

Good rules refer to other security groups instead of address ranges where possible, so they follow the workloads. Remove rules that nobody can explain.

Control outbound traffic

Most teams filter incoming traffic carefully and leave outgoing traffic open. Attackers rely on that: they download tools and send data out through ports that are never checked. Limit egress to the destinations a workload needs, through a proxy or a firewall with an allow list, and log denied attempts. Block access to the cloud metadata address from application code where you can.

Protect the edge

- **TLS everywhere**: modern versions only, strong ciphers, automatic certificate renewal, and HSTS.
- **Web application firewall**: blocks common attack patterns and lets you add virtual patches while a fix is prepared. It is a layer, not a replacement for secure code.
- **DDoS protection**: use the provider's absorbing services and a content delivery network, set rate limits, and plan capacity. Have a runbook and contacts ready before an attack.
- **Origin protection**: make the origin accept traffic only from the edge network, or the protection can be bypassed.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Defence layers from the internet to the database

Part	What it does	What goes wrong
Internet	The source of normal users and of attacks.	Nothing to fix here; assume hostile traffic.
CDN and DDoS shield	Absorbs floods, caches content and hides the origin address.	If the origin address leaks, attackers go around the shield.
WAF and load balancer	Terminates TLS, filters attacks and applies rate limits.	If rules are stale, new attacks pass. If it fails closed without a plan, users see an outage.
Application tier	Accepts traffic only from the load balancer security group.	With broad inbound rules, any host can reach it.
Egress proxy	Allows only approved outbound destinations and logs the rest.	Without it, a compromised app can send data anywhere.

Flow: Internet to CDN and DDoS shield / CDN and DDoS shield to WAF and load balancer / WAF and load balancer to Application tier / Application tier to Egress proxy

DNS and certificates

DNS is easy to forget and attractive to attackers.

- Protect registrar and DNS accounts with strong multi-factor authentication, because control of DNS is control of your traffic and mail.
- Watch for **dangling records**: a name that points to a cloud resource that was deleted can be claimed by someone else, which is a subdomain takeover.
- Use DNSSEC where it fits, and set mail records (SPF, DKIM and DMARC) to stop others sending as your domain.
- Monitor certificate transparency logs for certificates issued for your names, and alert on expiry well in advance.

```
$ dig +short CNAME promo.example.com
promo-site.azurewebsites.net.
$ dig +short promo-site.azurewebsites.net
$
```

An empty answer behind a CNAME suggests that the target no longer exists. Delete the record, or re-create the target, before somebody else claims it.

Private connectivity

Reach cloud services such as storage, queues and registries through private endpoints, so traffic stays off the internet and you can restrict access to your network. Use VPNs or private links for access between sites and clouds, and keep administration interfaces on private addresses behind strong authentication.

Incident: the open database port

> **Incident:** *A scan from outside shows that port 5432 of a database host answers from the internet. The security group was opened "for a quick test" and never closed.*

```
$ aws ec2 describe-security-groups --group-ids sg-0abc --query 'SecurityGroups[].IpPermissions[]'
[{"FromPort": 5432, "ToPort": 5432, "IpRanges": [{"CidrIp": "0.0.0.0/0"}]}]
```

Close the rule at once, check the database logs for logins from unknown addresses, rotate the database credentials, and add a policy check that fails any pull request or live change opening database ports to the world.

Common mistakes

- Allowing all outbound traffic from servers
- Security group rules that nobody owns or can explain

MARGA by EknathaLabs
marga.eknathalabs.com

- Dangling DNS records after resources are deleted
- Protecting the edge but leaving the origin reachable directly

Check yourself

Why restrict outbound traffic from servers?

Answer: An attacker who gains access cannot easily download tools or send data to their own server

What is a dangling DNS record?

Answer: A name that points to a resource that no longer exists, which someone else may be able to claim

Why must the origin accept traffic only from the edge network?

Answer: Otherwise attackers can send traffic straight to the origin and skip the protection

What does a private endpoint provide?

Answer: Access to a cloud service through private addresses, without crossing the public internet

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Ransomware resilience, backups and recovery

15 minutes. Understand how ransomware attacks unfold, design immutable and isolated backups, rehearse recovery, and reduce the blast radius so one compromised account cannot encrypt everything.

Ransomware is a business problem more than a technical curiosity. Attackers get in, spend days or weeks learning the environment, delete or encrypt backups, and then lock the systems that the business needs, often after copying data to threaten its release. The strongest defence is not a single product. It is the ability to restore from copies that the attacker could not touch, and a design that limits how far one stolen account can go.

What you will learn

- The typical stages of a ransomware attack
- The backup rule of 3-2-1 and why immutability matters
- Isolating backups and protecting their credentials
- Rehearsing recovery and measuring recovery time
- Reducing the blast radius before an attack happens

How an attack unfolds

1. **Initial access**: a phished login, an exposed remote desktop, an unpatched internet-facing system or a stolen token.
2. **Discovery and privilege escalation**: the attacker maps the network, finds administrator credentials and the backup systems.
3. **Preparation**: they disable security tools, delete snapshots and backups, and copy sensitive data out.
4. **Impact**: encryption of servers and databases at the same moment, often at night or at a weekend.
5. **Extortion**: a demand for payment, with a threat to publish the stolen data.

Defences can break the chain at every step, and the backup stage decides how bad the outcome is.

Backups that survive an attack

The common guideline is **3-2-1**: three copies of the data, on two kinds of media or services, with one copy off-site. For ransomware add two more ideas: one copy must be **immutable**, meaning it cannot be changed or deleted for a set time even by an administrator, and one copy must be **isolated**, meaning in a separate account or system with separate credentials.

```
# Object storage with a lock: objects cannot be deleted or overwritten for 30 days
aws s3api put-object-lock-configuration \
  --bucket acme-backups-locked \
  --object-lock-configuration '{"ObjectLockEnabled":"Enabled","Rule":{"DefaultRetention":{"Mode":" \
    COMPLIANCE","Days":30}}}'
```

In compliance mode nobody, including the account owner, can shorten the retention. That is exactly what you want when an attacker has the keys.

Protect the backup system itself

- Separate credentials and multi-factor authentication for backup administration, not shared with the main directory.
- Network isolation: the backup network should not be reachable from ordinary workstations.
- Alert on backup deletions, retention changes and failed jobs.
- Encrypt backups, and keep the keys safe and separate, with their own recovery plan.

MARGA by EknathaLabs marga.eknathalabs.com

Recovery you have practised

A backup that was never restored is a hope, not a plan. Define a recovery time objective (RTO) and a recovery point objective (RPO) for each system, and test them.

```
$ ./restore-test.sh --system billing-db --point "2 hours ago" --target scratch
restore started 10:02:11
restore finished 10:47:40 (45m 29s)
verification: 18 of 18 checks passed
```

Write down the order of recovery, because systems depend on each other: identity and DNS first, then databases, then applications. Keep the recovery runbook and contacts offline, since the system that stores them may be encrypted too.

Diagram: Recovery order after a destructive attack

Part	What it does	What goes wrong
Identity and DNS	Directory, authentication and name resolution. Everything else needs them.	If they are down or compromised, nothing else can be trusted or started.
Core network	Routing, firewalls and access for responders.	Rebuilt network rules must be clean, or the attacker may return.
Databases	Restored from immutable backups to a known good point.	Restoring too late a point brings back the malware; too early loses data.
Applications	Rebuilt from signed images and code, not repaired in place.	Restoring infected binaries restarts the attack.
Users and traffic	Returned in stages after checks.	Opening everything at once can overload recovery and hide a second wave.

Flow: Identity and DNS to Core network / Identity and DNS to Databases / Core network to Applications / Databases to Applications / Applications to Users and traffic

Reduce the blast radius

- **Least privilege and tiered administration**: separate accounts for daily work and for administration, and none with rights everywhere.
- **Segmentation**: stop one infected workstation from reaching servers directly.
- **Patch and harden** internet-facing systems, and close remote desktop to the internet.
- **Endpoint detection** to stop encryption early, and alerts for mass file changes.
- **Immutable infrastructure**: servers that you rebuild from code are much easier to recover than hand-built ones.

Incident: backups deleted first

> **Incident:** *On a Saturday, monitoring shows hundreds of snapshot deletions in the cloud account, followed by encrypted virtual machines. The attacker used a stolen administrator token.*

```
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=DeleteSna \
    pshot --max-results 3
{"Events": [{"EventName": "DeleteSnapshot", "Username": "admin-ci", "EventTime": "2026-10-17T02:14 \
    :09Z"}]}
```

Contain first: disable the token, isolate affected hosts and block the address. Then restore from the copy in the separate account that has a retention lock. After recovery, move to short-lived credentials, remove delete rights from day-to-day roles, and add an alert on mass deletion events.

Common mistakes

- Keeping backups reachable with the same credentials as production
- Never testing a restore, or testing only a small sample
- Storing the recovery plan only on systems that may be encrypted

MARGA by EknathaLabs
marga.eknathalabs.com

- Paying without checking whether the data can be restored from backup

Check yourself

What does an immutable backup protect against?

Answer: Deletion or change by an attacker, even one with administrator credentials, until the retention period ends

Why keep backups in a separate account with separate credentials?

Answer: A stolen production login cannot reach them

What should you do regularly with backups?

Answer: Restore them in a test and measure the time and the result

In what order should recovery usually begin?

Answer: Identity and DNS, then network and databases, then applications

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 13. Threat detection and security logging

14 minutes. Decide what to log, centralise and protect it, write detections that matter, reduce false positives, and use frameworks such as ATT&CK to find coverage gaps.

Prevention fails sometimes, and the question then is how fast you notice. Many breaches last months before discovery, because logs were missing, scattered or never looked at. Detection engineering turns logs into signals that a small team can act on.

What you will learn

- What to log in cloud, Kubernetes, hosts and applications
- Protecting and centralising logs
- Writing detection rules and testing them
- Reducing noise and handling alerts
- Mapping coverage with ATT&CK

What to log

Source	Events worth keeping
Cloud control plane	Logins, role assumption, policy changes, key creation, resource deletion, public exposure
Identity provider	Sign-ins, failures, multi-factor changes, new devices, impossible travel
Kubernetes	API audit log: exec into Pods, secret reads, RBAC changes, privileged Pods
Hosts and containers	Process start, privilege escalation, new users, network connections, file changes in sensitive paths
Network	DNS queries, flow logs, proxy and firewall decisions
Applications	Authentication, authorisation failures, admin actions, input validation errors
CI/CD	Pipeline changes, secret use, who approved a deployment

Include who, what, where from, when, and result. Use synchronised clocks and a common format with consistent field names.

Protect the logs

Attackers delete logs. Send them off the system quickly to a central store in a separate account that workload administrators cannot change. Make them tamper-evident (object lock, hash chains, write-once storage), restrict who can read them (logs contain sensitive data), and set retention that satisfies investigations and rules (often a year or more for audit logs).

Detections

Good detections look for behaviour that is rare and meaningful for your environment:

- A root or break-glass account signs in
- A new access key is created for a user who has never had one
- A role is assumed from a country or network never seen before
- Audit logging is disabled or a log bucket policy changes
- A Pod is started with host networking, or a shell runs in a production container
- Many failed logins followed by a success

MARGA by EknathaLabs marga.eknathalabs.com

- Large data download by a service that normally uploads

```
title: Cloud audit logging disabled
logsource: { product: cloud, service: audit }
detection:
  selection:
    eventName: [StopLogging, DeleteTrail, UpdateTrail]
  condition: selection
level: high
```

Rules written in a shared format (such as Sigma) can be converted for your tools. Keep them in version control with tests: a sample of the event that should fire, and events that should not.

Fewer false alarms

Alerts nobody trusts are worse than none. Tune rules with allow lists for known automation, attach context (who, from where, what changed, links to the user's recent activity), rate-limit repeats, and track precision: of the alerts last month, how many were real? Retire rules that never produce action. Prioritise by impact: a new admin on a production account beats a failed login.

Response links

Each detection needs a short runbook: what it means, how to confirm, how to contain, and who to call. Practise with simulated events (a canary access key whose use alerts immediately is cheap and effective, and so is a canary file or database record).

Coverage with ATT&CK

MITRE ATT&CK catalogues attacker techniques by stage. Map your detections to techniques and look for stages with none (for example persistence, credential access, exfiltration). Use it as a map for gaps, not as a score to maximise.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Threat detection and security logging fits together

Part	What it does	What goes wrong
What to log	Include who, what, where from, when, and result.	Logs stored where an attacker can delete them.
Protect the logs	Attackers delete logs. Send them off the system quickly to a central store in a separate account that workload administrators cannot change.	Logging that is on but never read.
Detections	Good detections look for behaviour that is rare and meaningful for your environment.	Detections without runbooks.
Fewer false alarms	Alerts nobody trusts are worse than none.	Counting rules rather than measuring useful alerts.
Response links	Each detection needs a short runbook: what it means, how to confirm, how to contain, and who to call.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Coverage with ATT&CK	MITRE ATT&CK catalogues attacker techniques by stage.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: What to log to Protect the logs / Protect the logs to Detections / Detections to Fewer false alarms / Fewer false alarms to Response links / Response links to Coverage with ATT&CK;

Incident: found after seven months

Incident: *An external researcher reports that a company's storage bucket was public, and investigators cannot tell who accessed it.*

MARGA by EknathaLabs marga.eknathalabs.com

```
$ aws s3api get-bucket-logging --bucket customer-exports
{}
$ aws cloudtrail describe-trails --query 'trailList[].Name'
[]
```

Neither data-event logging nor audit trails were enabled, so there was no record of access. The company enabled organisation-wide audit logs into a locked account, turned on object-level logging for sensitive buckets, added detections for public exposure, and placed canary tokens in key data stores.

Common mistakes

- Logging that is on but never read
- Logs stored where an attacker can delete them
- Detections without runbooks
- Counting rules rather than measuring useful alerts

Check yourself

Why ship logs to a separate account quickly?

Answer: An attacker who controls a system can erase local logs

Which event is worth an alert in a cloud account?

Answer: Audit logging being disabled

How do you reduce false positives?

Answer: Add context, tune with known automation and measure precision

What is ATT&CK mainly useful for here?

Answer: Finding gaps in detection coverage by attacker technique

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 14. Encryption and key management in practice

14 minutes. Use encryption at rest, in transit and in use correctly, manage keys with a key management service, rotate and separate duties, and avoid the common mistakes that make encryption useless.

Encryption is often treated as a checkbox: "encrypted at rest: yes". But encryption only protects against specific threats, and its strength rests entirely on how keys are managed. A strongly encrypted database whose key sits next to it in a configuration file is protected against almost nothing. This lesson shows how to use encryption so that it really reduces risk.

What you will learn

- What encryption at rest, in transit and in use protect against
- Symmetric and asymmetric keys, hashing and signing in plain terms
- Key management services and envelope encryption
- Rotation, access control and separation of duties
- Common mistakes

What it protects against

Kind	Protects against	Does not protect against
At rest (disks, databases, object storage, backups)	Lost or stolen disks, access to raw storage, some cloud-side mistakes	An attacker using valid application credentials, who sees decrypted data
In transit (TLS between client and server, and between services)	Eavesdropping and tampering on the network	A compromised endpoint
In use (confidential computing)	A compromised host or hypervisor reading memory	Application bugs
Application-level (field encryption)	Database administrators and anyone with database access	Compromise of the application's key

Choose by threat. Encrypt everything at rest by default because it is cheap, but know that it is mainly a compliance and basic-hygiene control.

Building blocks

- **Symmetric encryption** (such as AES-256-GCM) uses one key to encrypt and decrypt; fast, used for data.
- **Asymmetric encryption** uses a public key to encrypt or verify, and a private key to decrypt or sign; used to exchange keys and to prove identity (certificates, signatures).
- **Hashing** produces a fixed fingerprint that cannot be reversed; use slow, salted password hashing (Argon2, bcrypt) for passwords, never plain SHA-256.
- **Signing** proves who produced data and that it was not changed (used for images, packages and tokens).
- **Never invent your own algorithms or modes**; use well-reviewed libraries with safe defaults.

Key management and envelope encryption

Use a key management service (a cloud KMS, or a hardware-backed vault) so that keys are created, stored and used inside a protected service. With **envelope encryption**, a data key encrypts the data, and a master key in the KMS encrypts the data key. Applications ask the KMS to decrypt the data key, so the master key never leaves the service and every use is logged and controlled by policy.

```
$ aws kms generate-data-key --key-id alias/orders --key-spec AES_256
# returns a plaintext key (use, then discard from memory) and an encrypted copy (store next to the \
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
    data)
$ aws kms decrypt --ciphertext-blob fileb://datakey.enc
```

Access and separation of duties

- Key policy decides who can use a key and who can administer it; keep these roles separate. The person who manages keys should not be able to read the data, and the application that reads data should not be able to delete keys.
- Grant `Decrypt` only to the specific roles that need it; deny key deletion except through a reviewed process, and use a waiting period before deletion.
- Use different keys for different data classes and environments, so one leak does not expose everything.
- Log every key use and alert on unusual patterns.

Rotation and recovery

Rotate keys on a schedule and after suspected exposure. Managed services rotate the master key material automatically while old versions keep decrypting old data. Plan for loss: deleting a key makes data permanently unreadable, which is sometimes the goal (crypto-shredding) and sometimes a disaster. Back up what you must, test recovery, and protect the backup keys.

TLS everywhere

Use TLS 1.2 or newer for all connections, including inside the network. Automate certificate issue and renewal and alert on expiry. Verify certificates properly in clients; never turn verification off to "make it work".

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Encryption and key management in practice fits together

Part	What it does	What goes wrong
What it protects against	Choose by threat. Encrypt everything at rest by default because it is cheap, but know that it is mainly a compliance and basic-hygiene control.	Keys stored with the data they protect.
Building blocks	Symmetric encryption (such as AES-256-GCM) uses one key to encrypt and decrypt; fast, used for data.	- Hashing produces a fixed fingerprint that cannot be reversed; use slow, salted password hashing (Argon2, bcrypt) for passwords, never...
Key management and envelope	Use a key management service (a cloud KMS, or a hardware-backed vault) so that keys are created, stored and used inside a protected service.	Treating "encrypted at rest" as protection against all attackers.
Access and separation of duties	Key policy decides who can use a key and who can administer it; keep these roles separate.	One key for everything.
Rotation and recovery	Rotate keys on a schedule and after suspected exposure.	Check the Common mistakes list below this diagram for what usually goes wrong here.
TLS everywhere	Use TLS 1.2 or newer for all connections, including inside the network.	Disabling certificate verification.

Flow: What it protects against to Building blocks / Building blocks to Key management and envelope / Key management and envelope to Access and separation of duties / Access and separation of duties to Rotation and recovery / Rotation and recovery to TLS everywhere

MARGA by EknathaLabs marga.eknathalabs.com

Incident: the key in the repository

> **Incident:** *A security scan finds an encryption key in a configuration file in the same repository as the encrypted exports.*

```
$ grep -R "AES_KEY" config/
config/prod.env: AES_KEY=9f2c...e1
$ aws s3 ls s3://exports-prod/ | head -1
2026-09-30 exports-2026-09.enc
```

Anyone with access to the repository could decrypt every export. The team moved to envelope encryption using a KMS key with a narrow policy, rotated the exposed key, re-encrypted past exports, removed the key from history, and added a secret scanner to the pipeline.

Common mistakes

- Keys stored with the data they protect
- One key for everything
- Disabling certificate verification
- Treating "encrypted at rest" as protection against all attackers

Check yourself

What does encryption at rest mainly protect against?

Answer: Access to raw storage such as stolen disks or backups

What is envelope encryption?

Answer: A data key encrypts the data and a master key in a KMS encrypts the data key

Why separate key administrators from data readers?

Answer: So no single person can both control keys and read the data

What should you use to store passwords?

Answer: A slow, salted hash such as Argon2 or bcrypt

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 15. Supply chain security: SBOMs, signing and provenance

13 minutes. Know what is inside your software, prove where it was built, and verify signatures before anything runs in production.

A popular library you depend on is found to contain a serious flaw. The first question is simple and often unanswerable: which of our services use it, and in which version? Supply chain security is about being able to answer that in minutes, and about trusting only artifacts you can trace back to a build you control.

What you will learn

- What a software bill of materials (SBOM) is and how to generate one
- Signing artifacts and verifying signatures
- Provenance: proof of how and where an artifact was built
- Practical controls: pinning, lockfiles, private mirrors and admission checks

Know what you ship

An **SBOM** is a machine-readable list of the components inside an artifact: libraries, versions, licences and where they came from. Common formats are SPDX and CycloneDX. Generate one at build time for every image and package, store it next to the artifact, and index them so you can search across all services.

```
$ syft registry.example.com/shop/api:1.8.2 -o cyclonedx-json > api-1.8.2.sbom.json
$ grype sbom:api-1.8.2.sbom.json --fail-on high
```

When a new vulnerability is announced, you search the SBOM index for the affected package and version, not every repository by hand. An SBOM does not make software safe; it makes questions answerable.

Sign and verify

A signature proves that an artifact came from a known identity and was not changed afterwards. Keyless signing with short-lived certificates tied to the pipeline identity avoids long-lived private keys.

```
$ cosign sign registry.example.com/shop/api@sha256:9f2c...
$ cosign verify registry.example.com/shop/api@sha256:9f2c... \
    --certificate-identity-regexp "https://ci.example.com/.*" \
    --certificate-oidc-issuer https://ci.example.com
```

Signing alone is worthless unless something checks it. Verify in the cluster with an admission policy that refuses unsigned images, and refer to images by digest, because tags can be moved.

Provenance

Provenance is a signed statement of how an artifact was made: which source revision, which builder, which build steps and inputs. The SLSA framework describes levels: at the lowest you have a build script, at higher levels the build runs on a hosted, isolated builder that produces provenance the developers cannot forge. With provenance you can answer "was this image built from the reviewed main branch by our pipeline?" and reject anything built on a laptop.

Everyday controls

- Use **lockfiles** and pin versions, with hashes where the tool supports it.
- Pull dependencies through a **private mirror** that scans and caches them, so a deleted or hijacked public package does not break or poison builds.
- Watch for **typosquatting** (similar names) and for sudden maintainer changes.
- Update dependencies on a schedule with automated pull requests and tests, so security fixes are small.
- Keep build systems patched and short-lived, because the pipeline itself is a target.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Supply chain security: SBOMs, signing and... fits together

Part	What it does	What goes wrong
Know what you ship	An SBOM is a machine-readable list of the components inside an artifact: libraries, versions, licences and where they came from.	Pulling dependencies straight from the public internet in every build.
Sign and verify	A signature proves that an artifact came from a known identity and was not changed afterwards.	Signing images without verifying the signature anywhere.
Provenance	Provenance is a signed statement of how an artifact was made: which source revision, which builder, which build steps and inputs.	The SLSA framework describes levels: at the lowest you have a build script, at higher levels the build runs on a hosted, isolated builder...
Everyday controls	Use lockfiles and pin versions, with hashes where the tool supports it.	Generating SBOMs and never searching them.

Flow: Know what you ship to Sign and verify / Sign and verify to Provenance / Provenance to Everyday controls

Incident: Which services use this library?

Incident: *On the day a critical flaw in a logging library was announced, management asked for the list of affected services. It took the team three days.*

```
$ grep -r "log4j" */pom.xml | wc -l
41
$ ./sbom-search.sh log4j-core 2.14
no index found
```

Without SBOMs the team searched repositories, base images and vendor products by hand, and missed transitive dependencies. Afterward they generated SBOMs in every build, stored them with the artifacts, built a small search service, and added an admission policy that only allows signed images built by the pipeline. The next announcement was answered in twenty minutes.

Common mistakes

- Generating SBOMs and never searching them
- Signing images without verifying the signature anywhere
- Using tags instead of digests
- Pulling dependencies straight from the public internet in every build

Check yourself

What is an SBOM?

Answer: A machine-readable list of the components inside software

Why verify signatures at deploy time?

Answer: A signature only helps if something refuses unsigned or untrusted artifacts

What does provenance describe?

Answer: How, where and from which source an artifact was built

Why refer to images by digest?

Answer: A tag can be moved to different content, a digest cannot

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 16. Cloud security incident playbooks

13 minutes. Prepare short playbooks for the common cloud incidents: leaked keys, public storage and compromised workloads.

At 02:00 an alert says an access key from your account is being used from another continent. You have minutes before data leaves. A playbook turns panic into steps: contain, preserve evidence, remove access, recover and learn. Writing it now costs an afternoon, and writing it during the incident costs the night.

What you will learn

- The phases of cloud incident response
- Playbook: leaked access key
- Playbook: publicly exposed storage
- Playbook: compromised instance or container, and the habits that help

Phases

1. **Detect and triage**: is it real, how large, what is affected?
2. **Contain**: stop the damage without destroying evidence.
3. **Preserve evidence**: snapshots, logs, memory if possible.
4. **Eradicate and recover**: remove the attacker, rotate secrets, restore clean systems.
5. **Learn**: blameless review and fixes.

Decide in advance who leads, who communicates, and who may take disruptive actions such as disabling a production role. Keep a separate, trusted account or channel for the response team in case the main one is compromised.

Playbook: leaked access key

```
$ aws iam update-access-key --user-name ci-bot --access-key-id AKIA... --status Inactive
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=AccessKeyId,AttributeValue=AKIA... \
    \
    --max-results 50
$ aws iam list-user-policies --user-name ci-bot
```

Deactivate (do not delete yet) the key, so you keep the record. Search the audit log for everything the key did: new users, new keys, changed policies, data reads, new compute. Revoke sessions that were created from it. Check for persistence, such as new roles or access keys, and remove them. Then rotate any secret that the key could read. Finally, find out how it leaked, for example a public repository or a log file.

Playbook: exposed storage

Block public access at once, then work out the exposure. Turn on the account-level block, check access logs for who read what and when, and decide whether the data was sensitive. If personal data was exposed, legal and privacy teams need to know early because deadlines may apply. Fix the cause (a template that allowed public buckets) and add a preventive control, such as a policy that denies creating public buckets.

Playbook: compromised workload

Isolate rather than delete: change the security group to deny all traffic except from the investigation host, or apply a deny-all network policy to the pod. Snapshot the disk, collect logs, and keep the instance running if memory evidence matters. Remove its role permissions. Replace the workload from a known-good image, then investigate offline. Look at what the workload's role could reach, because that defines the possible damage.

MARGA by EknathaLabs
marga.eknathalabs.com

Habits that help: central immutable audit logs, short-lived credentials, tested break-glass access, and a practice incident each quarter.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Cloud security incident playbooks fits together

Part	What it does	What goes wrong
Phases	Detect and triage: is it real, how large, what is affected?	Deleting evidence while containing.
Playbook: leaked access key	Deactivate (do not delete yet) the key, so you keep the record.	No written playbook or owner.
Playbook: exposed storage	Block public access at once, then work out the exposure.	Not knowing who may disable production access.
Playbook: compromised workload	Isolate rather than delete: change the security group to deny all traffic except from the investigation host, or apply a deny-all network policy to the pod.	Habits that help: central immutable audit logs, short-lived credentials, tested break-glass access, and a practice incident each quarter.

Flow: Phases to Playbook: leaked access key / Playbook: leaked access key to Playbook: exposed storage / Playbook: exposed storage to Playbook: compromised workload

Incident: Keys in a public repository

> **Incident:** *A developer pushed a script with a long-lived key to a public code-sharing site. Within 20 minutes the account had 40 new compute instances mining coins.*

```
$ aws ec2 describe-instances --filters Name=instance-state-name,Values=running --query "length(Res \
    ervations[].Instances[])"
43
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=RunInstan \
    ces --max-results 3
```

The on-call engineer followed the playbook: key deactivated in four minutes, instances stopped and snapshotted, and CloudTrail searched for new users and roles. A rogue access key on a second user was found and removed. The company replaced long-lived keys with short-lived role credentials, added secret scanning before commits, and set budget alarms that page on sudden spend.

Common mistakes

- No written playbook or owner
- Deleting evidence while containing
- Rotating one key and forgetting the persistence the attacker added
- Not knowing who may disable production access

Check yourself

What should you do first with a leaked key?

Answer: Deactivate it and review its activity in the audit log

Why isolate a compromised instance instead of deleting it?

Answer: You keep evidence for investigation while stopping the harm

What prevents the exposed storage problem in the future?

Answer: A preventive policy that denies public buckets, plus account-wide blocking

Which credentials reduce the impact of leaks?

Answer: Short-lived role credentials

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 17. Investigating security alerts and failed controls

16 minutes. Triage an alert, preserve evidence, trace a suspicious action through audit logs and contain it, and debug security controls that block legitimate work.

Security debugging has two sides. One is investigating something suspicious: was that login or API call really an attack? The other is debugging a control that works too well: a policy that blocks a valid deploy. In both, you follow evidence in logs, keep your own actions from destroying it, and change as little as possible.

What you will learn

- Triaging an alert: true positive, false positive or unknown
- Preserving evidence before containment
- Tracing an action through audit and access logs
- Debugging failed admission, policy, TLS and permission checks
- Containment steps that are reversible

Triage questions

1. What exactly fired, and what data did the rule use?
2. Who, from where, to what, and when? (identity, source address, resource, time)
3. Is this normal for that identity? Compare with the last 30 days.
4. What is the worst case if this is real, and how fast can it spread?

Trace the action

```
# Who used this access key, and what did it do?
aws cloudtrail lookup-events --lookup-attributes AttributeKey=AccessKeyId,AttributeValue=AKIAEXAMP \
    LE --max-results 20
# Failed logins from one address
grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | hea \
    d
# Kubernetes audit: who deleted this?
jq 'select(.verb=="delete" and .objectRef.name=="payments")' audit.log
```

Build a timeline: first sign, what the actor touched, and the last sign. Check for persistence such as new users, keys, scheduled jobs and changed policies.

Preserve, then contain

Containment should be reversible and should not destroy evidence. Prefer isolating a host with a restrictive security group to powering it off, disable a key rather than delete it, snapshot disks before cleaning, and copy logs to a separate store. Rotate any credential that might have been exposed.

Debugging a control that blocks good work

Symptom	Where to look
Deploy rejected by admission policy	The denial message names the rule; test with a dry run
Image blocked by signature check	Was it signed by the expected identity? Is the policy up to date?
Scanner fails the build	Read the finding, check the fixed version, add a documented exception with an expiry
mTLS or certificate errors	Expiry, name, trust chain, clock skew
Permission denied after hardening	Audit log shows the denied call; add the narrowest allow

MARGA by EknathaLabs marga.eknathalabs.com

```
$ kubectl apply -f deploy.yaml --dry-run=server
Error from server: admission webhook "policy.example.com" denied the request: containers must not  \
    run as root
```

The message states the rule and the fix: set `runAsNonRoot: true`. Do not remove the policy to get past it.

Diagram: Security investigation flow

Part	What it does	What goes wrong
Alert or report	A detection rule or a person raises a suspicion.	Ignoring low alerts that repeat lets real attacks blend in.
Triage	Decide true, false or unknown using identity, source and history.	Calling it false without checking baselines.
Preserve evidence	Snapshot disks, copy logs, record the timeline.	Wiping a host first destroys what you need.
Contain	Disable keys, isolate hosts, block addresses, reversibly.	Deleting resources removes both evidence and the ability to undo.
Eradicate and recover	Remove persistence, rotate secrets, restore from known-good.	Restoring while the attacker's access remains.

Flow: Alert or report to Triage / Triage to Preserve evidence / Triage to Contain / Preserve evidence to Contain / Contain to Eradicate and recover

Incident: the build that was blocked, and the one that was not

Incident: *A new policy blocks all deployments from one team, and the same week a leaked token is used from an unknown address.*

```
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=Username,AttributeValue=ci-deploy  \
    --max-results 3
ConsoleLogin? no   SourceIPAddress: 198.51.100.77   EventName: ListSecrets   09:41
```

The block was a signature rule that expected the old build identity, a configuration fix. The leaked token was real: it came from a build log that printed environment variables. The team disabled and rotated the token, reviewed its activity, masked secrets in logs, and moved to short-lived credentials from workload identity.

Common mistakes

- Powering off or reimaging a host before taking evidence
- Deleting a credential instead of disabling it first
- Removing a control to unblock a team
- Closing an alert without checking a baseline

Check yourself

What should you do first when you find a leaked access key?

Answer: Disable it and review its recent activity in the audit log

Why preserve evidence before cleaning a compromised host?

Answer: So you can learn what the actor did and find persistence

A policy denies a deployment with "must not run as root". What is the right fix?

Answer: Change the workload to run as a non-root user

Which containment action is usually preferred first?

Answer: Isolate the host with a restrictive network rule

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 18. Web and API attacks every operator should know

13 minutes. Recognize injection, broken access control, SSRF and abuse of APIs from logs and defend with input handling, authorization checks and edge controls.

You do not need to write exploits to defend against them. Knowing how the common attacks work tells you what to look for in logs, what to harden and what to ask of the code.

What you will learn

- Injection and parameterized queries
- Broken access control and IDOR
- Server-side request forgery and cloud metadata
- Logs, rate limits and edge controls

Injection

```
# unsafe: user input becomes SQL
cur.execute(f"SELECT * FROM users WHERE email = '{email}'")

# safe: parameters are data, never code
cur.execute("SELECT * FROM users WHERE email = %s", (email,))
```

In logs it looks like requests containing quotes, `UNION SELECT`, `../` or shell characters. Prevent it with parameterized queries, input validation and least-privilege database accounts.

Broken access control

The most common serious flaw is a missing check that a user may access an object. In an **IDOR**, changing `/api/orders/1041` to `/api/orders/1042` returns someone else's order. Every request must check on the server that the caller is allowed to access that object. Look for one account requesting many sequential identifiers.

SSRF

A feature that fetches a URL for the user, such as image import or webhooks, can be abused to reach internal services and the cloud metadata address `169.254.169.254`.

```
# require the metadata token on AWS and block links that point to it from workloads
aws ec2 modify-instance-metadata-options --instance-id i-0abc --http-tokens required --http-put-re \
    sponse-hop-limit 1
```

Validate destinations against an allowlist, block private and link-local ranges after DNS resolution, and limit outbound traffic with egress rules.

Operate

```
awk '$9==401||$9==403 {print $1}' access.log | sort | uniq -c | sort -nr | head
grep -E "(union.*select|\.\./|<script)" -i access.log | head
```

Apply rate limits, a web application firewall in front of the application, strong authentication and monitoring of authorization failures. Treat repeated 401 and 403 responses from one source as a signal.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Web and API attacks every operator should know fits together

Part	What it does	What goes wrong
Injection	In logs it looks like requests containing quotes, UNION SELECT, ../ or shell characters.	Building queries by string concatenation.
Broken access control	The most common serious flaw is a missing check that a user may access an object.	Checking authentication but not object ownership.
SSRF	A feature that fetches a URL for the user, such as image import or webhooks, can be abused to reach internal services and the cloud metadata address 169.254.169.254.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Operate	Apply rate limits, a web application firewall in front of the application, strong authentication and monitoring of authorization failures.	No alerts on repeated authorization failures.

Flow: Injection to Broken access control / Broken access control to SSRF / SSRF to Operate

Incident: Sequential order numbers

Incident: *One account downloaded thousands of other customers' orders.*

```
GET /api/orders/20411 200 user=acct-882
GET /api/orders/20412 200 user=acct-882
3400 sequential requests in 9 minutes
```

The API checked that the user was logged in but not that the order belonged to them. A log alert on one account reading many identifiers caught it. The team added an ownership check in the data layer, tests for object access and per-account rate limits.

Common mistakes

- Checking authentication but not object ownership
- Building queries by string concatenation
- Letting servers fetch any URL
- No alerts on repeated authorization failures

Check yourself

What prevents SQL injection best?

Answer: Parameterized queries

What is an IDOR?

Answer: Accessing objects by changing an identifier

What can SSRF reach?

Answer: Internal services and metadata

What helps detect abuse?

Answer: Rate limits and log alerts

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 19. SAST, DAST and dependency scanning gates

13 minutes. Add static analysis, dependency scanning and dynamic testing to pipelines, tune them to reduce noise and decide what should block a release.

Automated security testing is easy to add and hard to run well. The difference lies in what runs when, what blocks and who owns the findings.

What you will learn

- SAST, SCA, DAST, secrets and IaC scanners
- Where each scan fits in the pipeline
- Tuning thresholds and suppressions
- Triage, ownership and measuring the program

The tools

- **SAST:** analyzes source code for flaws such as injection and unsafe functions.
- **SCA:** finds known vulnerabilities in third-party dependencies and licence issues.
- **Secrets scanning:** finds keys and tokens in code and history.
- **IaC and container scanning:** checks Terraform, Kubernetes manifests and images.
- **DAST:** probes a running application in a test environment for issues visible from outside.

In the pipeline

```
# example stages in a CI pipeline
stages:
  pr_checks:        # fast, under 5 minutes, runs on every pull request
    - secret_scan
    - sca_scan --fail-on critical,high --only-fixable
    - sast_scan --changed-files-only
    - iac_scan
  nightly:          # slower, deeper
    - sast_full
    - container_scan
    - dast_scan --target https://staging.example.com
```

Run quick scans where developers see results in the pull request. Run slow scans nightly and report to a backlog.

Thresholds

Block on findings that are **severe, fixable and reachable**. Report the rest with a deadline by severity, for example critical in 7 days and high in 30. Use time-limited suppressions with a reason and an owner, never permanent ignores.

```
trivy fs --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 .
```

Own and measure

Route each finding to the owning team, not a central queue. Track mean time to remediate, open findings by age and false positive rate per tool. Retire tools whose findings are mostly ignored, and review the rules every quarter.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How SAST, DAST and dependency scanning gates fits together

Part	What it does	What goes wrong
The tools	SAST: analyzes source code for flaws such as injection and unsafe functions.	Check the Common mistakes list below this diagram for what usually goes wrong here.
In the pipeline	Run quick scans where developers see results in the pull request.	Slow scans on every pull request.
Thresholds	Block on findings that are severe, fixable and reachable.	Permanent suppressions with no owner.
Own and measure	Route each finding to the owning team, not a central queue.	Blocking on every severity.

Flow: The tools to In the pipeline / In the pipeline to Thresholds / Thresholds to Own and measure

Incident: Everything blocked, everything suppressed

Incident: *A strict gate led to blanket suppressions that hid a real flaw.*

```
scan: 214 findings (3 critical, 41 high, 170 medium)
gate: fail on medium and above
suppressions added in one week: 198 (including 1 critical)
```

Developers needed to ship, so they suppressed most findings in bulk, including a real critical issue. The team changed the gate to block on fixable critical and high issues, required reasons and expiry dates for suppressions and reviewed them weekly.

Common mistakes

- Blocking on every severity
- Permanent suppressions with no owner
- Slow scans on every pull request
- No measurement of false positives

Check yourself

What does SCA scan?

Answer: Third-party dependencies

What should block a release?

Answer: Severe, fixable, reachable findings

Where run slow scans?

Answer: Nightly

What should a suppression have?

Answer: A reason, owner and expiry

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 20. Cloud posture and misconfiguration management

12 minutes. Find public buckets, open ports and missing encryption continuously with posture tools and policy as code, and fix drift at the source.

Cloud accounts change constantly, and every change is a chance to open something that should be closed. Continuous checks against known good settings find these mistakes while they are small.

What you will learn

- Common misconfigurations and how attackers find them
- Benchmarks, posture tools and IaC scanning
- Prioritizing by exposure and sensitivity
- Fixing at the source and preventing recurrence

Common problems

- Public storage buckets and snapshots
- Security groups open to the whole internet on admin ports such as 22 and 3389
- Unencrypted volumes and databases
- Overly broad roles and long-lived access keys
- Disabled audit logging
- Default credentials on managed services

Detect

```
aws s3api get-public-access-block --bucket shop-exports
aws ec2 describe-security-groups --filters Name=ip-permission.cidr,Values=0.0.0.0/0 \
  --query 'SecurityGroups[].{id:GroupId,ports:IpPermissions[].FromPort}'
checkov -d infra/ --framework terraform --quiet
```

Run posture checks continuously on live accounts, and scan infrastructure code in pull requests so mistakes never get deployed. Use a recognized benchmark such as **CIS** as a starting baseline.

Prioritize

Not all findings are equal. Rank by **internet exposure**, **data sensitivity**, **privilege** and **ease of exploitation**. A public bucket with customer data outranks a missing tag. Combine findings: a public instance with a powerful role and a known vulnerability is a critical chain.

Fix at the source

```
resource "aws_s3_bucket_public_access_block" "exports" {
  bucket                  = aws_s3_bucket.exports.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

Change the Terraform or template and redeploy, otherwise drift returns. Add organization-level guardrails that deny the risky action, and a policy check in CI. Measure time to remediate by severity and report trends to owners.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Cloud posture and misconfiguration management fits together

Part	What it does	What goes wrong
Common problems	Public storage buckets and snapshots.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Detect	Run posture checks continuously on live accounts, and scan infrastructure code in pull requests so mistakes never get deployed.	Scanning accounts but not infrastructure code.
Prioritize	Not all findings are equal. Rank by internet exposure, data sensitivity, privilege and ease of exploitation.	Ranking all findings equally.
Fix at the source	Change the Terraform or template and redeploy, otherwise drift returns.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Common problems to Detect / Detect to Prioritize / Prioritize to Fix at the source

Incident: The export bucket that was public

Incident: *A test bucket with customer exports was publicly readable.*

```
bucket: shop-exports-test
BlockPublicAcls: false, RestrictPublicBuckets: false
object customers_2026-10.csv read by 198.51.100.23
```

A developer created the bucket by hand and disabled the block settings for a quick test. A posture scan flagged it, the team closed it and rotated data exposure checks, then enforced an account-level public access block and a CI rule rejecting public buckets.

Common mistakes

- Fixing resources by hand and not in code
- Ranking all findings equally
- Scanning accounts but not infrastructure code
- No guardrail to prevent the mistake again

Check yourself

Why fix findings in code?

Answer: Hand fixes return at the next deploy

What raises a finding's priority?

Answer: Internet exposure and sensitive data

What is CIS used for?

Answer: A baseline of secure settings

What prevents public buckets across accounts?

Answer: An organization-level guardrail

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 21. Security champions and secure code review

12 minutes. Spread security through teams with champions, practical code review checklists, threat modeling and feedback loops that developers value.

A small security team cannot review every change. The scalable approach is to make security a normal part of how teams build, with a few trained people in each team and simple practices everyone can follow.

What you will learn

- What security champions do and what they do not
- A practical secure review checklist
- Lightweight threat modeling for new features
- Using findings to improve defaults

Champions

A **security champion** is a developer or operator in a product team who has a bit of time and training to help with security questions, reviews and triage. They are not responsible for all security, and they are not a gatekeeper. The program needs management support, a regular meeting, shared learning and a way to escalate to the security team.

A review checklist

Keep it short enough that people use it:

- **Authentication and authorization:** is every endpoint checked, including object ownership?
- **Input:** is all input validated, and queries parameterized?
- **Secrets:** nothing in code, logs or error messages?
- **Data:** is sensitive data minimized, encrypted and not logged?
- **Dependencies:** new packages reviewed, pinned and scanned?
- **Failure:** do errors reveal internals, and does failure fail closed?

```
## Security review (PR template)
- [ ] New or changed endpoints have authorization tests
- [ ] No secrets or personal data in code or logs
- [ ] New dependencies are pinned and scanned
- [ ] Threat model updated if data flow changed
```

Quick threat modeling

For each new feature, spend thirty minutes asking four questions: what are we building, what can go wrong, what will we do about it and did we do a good job. Draw the data flow, mark trust boundaries and list the top three risks with a mitigation each.

Improve the platform

Track what keeps recurring. If many findings are the same mistake, fix it in a shared library, template or pipeline check rather than in each service. Share incidents blamelessly as short stories, and celebrate good catches.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Security champions and secure code review fits together

Part	What it does	What goes wrong
Champions	A security champion is a developer or operator in a product team who has a bit of time and training to help with security questions, reviews and triage.	A champion program with no time or support.
A review checklist	Keep it short enough that people use it.	- Failure: do errors reveal internals, and does failure fail closed?
Security review (PR template)	[ ] New or changed endpoints have authorization tests.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Quick threat modeling	For each new feature, spend thirty minutes asking four questions: what are we building, what can go wrong, what will we do about it and did we do a good job.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Improve the platform	Track what keeps recurring. If many findings are the same mistake, fix it in a shared library, template or pipeline check rather than in each service.	Never fixing recurring issues at the platform level.

Flow: Champions to A review checklist / A review checklist to Security review (PR template) / Security review (PR template) to Quick threat modeling / Quick threat modeling to Improve the platform

Incident: Caught in review, not in a pentest

Incident: *A missing ownership check was found in code review.*

```
PR #2381: GET /exports/{id}
champion comment: where do we check the export belongs to the caller?
fixed before merge, test added
```

The champion used the checklist and noticed the missing check, which would have exposed other customers' files. The fix and test took minutes. The team then added a reusable ownership check helper to the shared library.

Common mistakes

- A champion program with no time or support
- Checklists too long to use
- Treating champions as gatekeepers
- Never fixing recurring issues at the platform level

Check yourself

What do champions do?

Answer: Help their team with security

Why keep checklists short?

Answer: People will actually use them

What does threat modeling ask?

Answer: What can go wrong and what we will do

How reduce repeat findings?

Answer: Fix them in shared libraries and templates

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 22. Incident forensics and evidence handling

13 minutes. Preserve volatile evidence, collect logs and disk images safely, keep a chain of custody and build a timeline during a security incident.

When a system is compromised, the instinct is to wipe it and restore service. A calm few steps first preserve the evidence you need to understand scope, close the hole and meet legal duties.

What you will learn

- The order of volatility and what to capture first
- Cloud evidence: snapshots, logs and access records
- Chain of custody and integrity
- Building a timeline and reporting

Order of volatility

Collect the most fleeting data first: running processes and network connections, memory if feasible, then logs and temporary files, then disk contents. On a live Linux host:

```
date -u; hostname
ps auxwwf > /evidence/ps.txt
ss -tunap > /evidence/connections.txt
last -a > /evidence/logins.txt
journalctl --no-pager --since "-48h" > /evidence/journal.txt
sha256sum /evidence/* > /evidence/SHA256SUMS
```

Write to a separate, protected location, not the compromised disk. Avoid running tools that change many files, and note every command you run.

Cloud evidence

```
aws ec2 create-snapshot --volume-id vol-0abc123 --description "IR-2026-1105 web-04 root"
aws ec2 modify-instance-attribute --instance-id i-0def456 --groups sg-quarantine
aws cloudtrail lookup-events --lookup-attributes AttributeKey=Username,AttributeValue=ci-deploy -- \
    max-results 50
```

Snapshot disks, **isolate** the instance with a quarantine security group instead of terminating it, and export audit logs and flow logs to a locked evidence bucket with retention.

Chain of custody

Record who collected what, when, from where and how, and hash each item. Store originals read-only and work on copies. This protects the integrity of evidence if legal or regulatory action follows.

Timeline

Merge events from cloud audit logs, authentication logs, host logs and network data into one UTC timeline: first access, actions taken, data touched and persistence. Mark what is confirmed and what is suspected. Involve legal and management early, and follow your notification obligations.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Incident forensics and evidence handling fits together

Part	What it does	What goes wrong
Order of volatility	Collect the most fleeting data first: running processes and network connections, memory if feasible, then logs and temporary files, then disk contents.	Writing evidence to the compromised disk.
Cloud evidence	Snapshot disks, isolate the instance with a quarantine security group instead of terminating it, and export audit logs and flow logs to a locked evidence bucket with...	Terminating or reimaging before collecting evidence.
Chain of custody	Record who collected what, when, from where and how, and hash each item.	Informing legal late.
Timeline	Merge events from cloud audit logs, authentication logs, host logs and network data into one UTC timeline: first access, actions taken, data touched and persistence.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Order of volatility to Cloud evidence / Cloud evidence to Chain of custody / Chain of custody to Timeline

Incident: Wiped before anyone looked

Incident: *A compromised server was reimaged immediately and evidence was lost.*

```
action: terminate instance i-0def456
disk snapshots: none
host auth logs: gone, data accessed: unknown
```

Restoring service was urgent, so the instance was terminated. Later the team could not prove which records the attacker saw, which affected notifications. Their new runbook quarantines first, snapshots disks and exports logs before any rebuild.

Common mistakes

- Terminating or reimaging before collecting evidence
- Writing evidence to the compromised disk
- Not hashing or documenting collection
- Informing legal late

Check yourself

What do you collect first?

Answer: Volatile data such as processes and connections

How should you isolate a cloud instance?

Answer: Quarantine group, not termination

What does a hash show?

Answer: The evidence was not altered

Why a timeline?

Answer: To understand what happened and in what order

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 23. Audit readiness and continuous compliance evidence

12 minutes. Prepare for SOC 2 or ISO 27001 audits by automating evidence, mapping controls to technical checks and keeping a living record.

Frameworks such as SOC 2 and ISO 27001 do not tell you how to configure a firewall. They ask whether you set rules, follow them and can show that you do. Engineers can answer that with data instead of documents.

What you will learn

- Controls, owners and technical checks
- Automating evidence collection
- Handling exceptions and drift
- Preparing for the auditor

From control to check

A **control** is a stated rule, such as "access is reviewed quarterly" or "changes are approved before production". Map each one to a **check** that can be run by a machine, an **owner** and the **evidence** it produces.

```
controls:
  - id: AC-2
    statement: Access to production is reviewed quarterly
    owner: platform-security
    check: scripts/access_review_report.py
    evidence: evidence/access-review/2026-Q4.json
  - id: CM-3
    statement: Production changes are reviewed and approved
    owner: engineering
    check: scripts/changes_without_review.py
    evidence: evidence/changes/2026-11.json
```

Collect automatically

```
python scripts/changes_without_review.py --since 30d > evidence/changes/2026-11.json
aws iam generate-credential-report >/dev/null && aws iam get-credential-report --query Content --o \
    utput text | base64 -d > evidence/iam/2026-11.csv
sha256sum evidence/*/*.json > evidence/SHA256SUMS
```

Run collection on a schedule from CI, store results in write-once storage with dates and hashes, and keep the scripts under version control. Evidence from real systems, such as pull request history, access logs and scan reports, is more convincing than screenshots.

Exceptions and drift

Failures happen. Record each exception with a reason, an approver, a mitigation and an end date. Alert when a control check starts failing, and fix it before the audit, not during it.

The audit

Give the auditor a map of controls to evidence, a named contact for each area and read-only access to the evidence store. Run an internal review a month before to find gaps. After the audit, turn findings into backlog items with owners.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Audit readiness and continuous compliance evidence fits together

Part	What it does	What goes wrong
From control to check	A control is a stated rule, such as "access is reviewed quarterly" or "changes are approved before production".	Collecting evidence only before the audit.
Collect automatically	Run collection on a schedule from CI, store results in write-once storage with dates and hashes, and keep the scripts under version control.	Controls with no owner.
Exceptions and drift	Failures happen. Record each exception with a reason, an approver, a mitigation and an end date.	Exceptions with no end date.
The audit	Give the auditor a map of controls to evidence, a named contact for each area and read-only access to the evidence store.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: From control to check to Collect automatically / Collect automatically to Exceptions and drift / Exceptions and drift to The audit

Incident: Screenshots in a panic

Incident: *Evidence gathering consumed weeks before an audit and still missed a control.*

```
AC-2 quarterly access review: no evidence for Q2
CM-3 changes: 14 production changes without a review record
audit fieldwork starts in 5 days
```

The team relied on memory and screenshots, so gaps appeared at the last minute. After mapping controls to automated checks with alerts and dated evidence, missed reviews were noticed within the quarter instead of at the audit.

Common mistakes

- Collecting evidence only before the audit
- Controls with no owner
- Exceptions with no end date
- Screenshots instead of system data

Check yourself

What is a control?

Answer: A stated rule with a way to prove it

Why automate evidence?

Answer: It is current, consistent and cheap

What should an exception include?

Answer: Reason, approver, mitigation and end date

What helps before the audit?

Answer: An internal review

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: DevSecOps and security

Scanning, secrets, signing, container and cluster hardening, cloud identity checks and incident first steps.

Scan code, dependencies and images

```
trivy fs --severity HIGH,CRITICAL --exit-code 1 .
trivy image --ignore-unfixed --severity HIGH,CRITICAL registry.example.com/app:1.4
trivy config ./terraform
checkov -d ./terraform
semgrep scan --config auto .
syft registry.example.com/app:1.4 -o spdx-json > sbom.json
grype sbom:./sbom.json --fail-on high
```

> **Caution:** *A clean scan means no known issues in the packages the tool recognised. It does not prove the application is safe.*

Secrets

```
trufflehog filesystem . --only-verified
detect-secrets scan > .secrets.baseline
aws sts get-caller-identity
aws iam list-access-keys --user-name deploy
aws iam update-access-key --user-name deploy --access-key-id AKIA... --status Inactive
```

> **Warning:** *Disable or rotate a leaked key first. Removing it from the repository does not make it safe.*

Sign and verify artifacts

```
cosign generate-key-pair
cosign sign --key cosign.key registry.example.com/app@sha256:abc123
cosign verify --key cosign.pub registry.example.com/app@sha256:abc123
cosign attest --predicate sbom.json --type spdxjson --key cosign.key registry.example.com/app@sha2 \
    56:abc123
```

Sign by digest, not by tag. A tag can move.

Harden containers

```
docker run --rm --read-only --tmpfs /tmp --cap-drop ALL \
  --security-opt no-new-privileges --pids-limit 200 --memory 256m app:1.4
docker run --rm --user 65532:65532 app:1.4
docker scout cves app:1.4
```

> **Danger:** *Do not use* `--privileged` *or mount* `/var/run/docker.sock` *into application containers. Both give control of the host.*

Kubernetes security checks

```
kubectl auth can-i --list -n shop --as system:serviceaccount:shop:web
kubectl get clusterrolebindings -o wide | grep cluster-admin
kubectl get pods -A -o jsonpath='{range .items[*]}{.metadata.namespace}{" "}{.metadata.name}{" "}{ \
    .spec.containers[*].securityContext.privileged}{"\n"}{end}'
kubectl label ns shop pod-security.kubernetes.io/enforce=restricted
kubectl get networkpolicy -A
kubectl get secret db -n shop -o jsonpath='{.data.password}' | base64 -d
```

> **Warning:** *The last command prints a secret to your terminal and shell history. Use it only when needed and clear the output.*

Cloud identity checks (AWS)

```
aws iam get-account-authorization-details > iam.json
aws iam generate-credential-report && aws iam get-credential-report
aws accessanalyzer list-findings --analyzer-arn ARN
aws s3api get-public-access-block --bucket NAME
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=ConsoleLogi \
    n
aws ec2 describe-security-groups --filters Name=ip-permission.cidr,Values=0.0.0.0/0
```

MARGA by EknathaLabs marga.eknathalabs.com

TLS and certificates

```
openssl s_client -connect example.com:443 -servername example.com </dev/null | openssl x509 -noout \
     -dates -issuer -subject
openssl x509 -in cert.pem -noout -text | head -30
nmap -sV --script ssl-enum-ciphers -p 443 example.com
```

Caution: *Scan only systems you own or have written permission to test.*

Incident first steps

```
ss -tunap | head -40
last -a | head
sudo journalctl -u sshd --since "1 hour ago" | grep -i "failed\|accepted"
sudo iptables -I INPUT -s 203.0.113.50 -j DROP
sudo tcpdump -i eth0 -w /secure/capture.pcap host 203.0.113.50
```

Danger: *Do not wipe or reboot a suspected compromised host before you have taken a snapshot or memory capture. You will destroy the evidence.*

Identity and tokens

```
echo "$TOKEN" | cut -d. -f2 | base64 -d 2>/dev/null | jq '{iss,aud,exp,scope}'
aws iam list-users --query 'Users[].UserName' --output text
aws iam get-user --user-name svc-old --query 'User.PasswordLastUsed'
```

Warning: *A JWT is signed, not encrypted. Never put secrets in it, and always check signature, issuer, audience and expiry.*

Application and API checks

```
curl -sI https://example.com | grep -iE 'strict-transport|content-security|x-content-type'
curl -s -o /dev/null -w "%{http_code}\n" https://api.example.com/invoices/10043 -H "Authorization: \
     Bearer $TOKEN"
```

Caution: *Test only your own systems. A 200 for another customer's object means object-level authorization is missing.*

Policy as code

```
opa eval -d policy/ -i plan.json "data.terraform.security.deny"
kubectl apply -f policy.yaml && kubectl get policyreport -A
```

Tip: *Start every new policy in audit mode, review violations with owners, then enforce in stages.*

Network and DNS

```
aws ec2 describe-security-groups --filters Name=ip-permission.cidr,Values=0.0.0.0/0 --query 'Secur \
    ityGroups[].GroupId'
dig +short CNAME promo.example.com
dig +short TXT example.com | grep -i spf
```

Caution: *A CNAME that points to a deleted cloud resource can be claimed by someone else. Remove dangling records.*

Backups and recovery

```
aws s3api put-object-lock-configuration --bucket backups --object-lock-configuration '{"ObjectLock \
    Enabled":"Enabled","Rule":{"DefaultRetention":{"Mode":"COMPLIANCE","Days":30}}}'
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=DeleteSnaps \
    hot --max-results 5
```

Danger: *Backups reachable with production credentials will be deleted first. Keep an immutable copy in a separate account and test restores.*

MARGA by EknathaLabs marga.eknathalabs.com